Séance 2 — De la donnée à l'objet

Sommaire

1. Classe et objet
   - 1.1 🟢 Pourquoi des classes
   - 1.2 🟢 Champs, constructeur, `this`
   - 1.3 🟢 Encapsulation et invariants
   - 1.4 🟢 Méthodes métier plutôt que setters
   - 1.5 🟢 `static` contre instance
2. `enum` et `record`
   - 2.1 🟢 `enum`
   - 2.2 🟢 `switch` sur un `enum`
   - 2.3 🔵 `enum` avec champs et méthodes ; le piège `ordinal()`
   - 2.4 🟢 `record`
   - 2.5 🟢 Le constructeur compact
   - 2.6 🔵 `record` ou classe ?
3. Types riches
   - 3.1 🟢 Le problème du `double`
   - 3.2 🟢 `BigDecimal` : créer et calculer
   - 3.3 🟢 `equals` contre `compareTo`, et la version finale de `Dish`
   - 3.4 🔵 Division et arrondis
   - 3.5 🟢 `LocalDateTime`
   - 3.6 🟢 `UUID`
   - 3.7 🔵 `Instant`, `Duration` et `Clock` : le temps des machines
4. Héritage
   - 4.1 🟢 Pourquoi l'héritage
   - 4.2 🟢 `extends`, `super(...)`, `protected`
   - 4.3 🟢 `@Override`, `abstract` et polymorphisme
   - 4.4 🟢 `Object`, la racine
   - 4.5 🟢 Composition plutôt qu'héritage
   - 4.6 🟣 Valider avant `super(...)` (JEP 513)
   - 4.7 🔵 Les limites de l'héritage
   - 4.8 🟢 Le patron de méthode : une règle que les sous-classes ne peuvent pas contourner
5. Interfaces et polymorphisme
   - 5.1 🟢 Le problème : une remise par type
   - 5.2 🟢 L'interface `Discount` et ses implémentations
   - 5.3 🟢 Polymorphisme
   - 5.4 🟢 Interface ou classe abstraite ?
   - 5.5 🔵 Méthodes `default` et `static` dans une interface
   - 5.6 🟢 `instanceof` avec motif : tester le type et nommer la variable
   - 5.7 🔵 `sealed interface` et `switch` avec motifs
6. Collections
   - 6.1 🟢 `List`, `Set`, `Map` sont des interfaces
   - 6.2 🟢 Utiliser les génériques

    - 6.3 🟢 `List`
    - 6.4 🟢 `Set`
    - 6.5 🟢 `Map` : `merge`, `getOrDefault`
    - 6.6 🟢 Le contrat `equals` / `hashCode` / `toString`
    - 6.7 🟢 `record` et `BigDecimal` : le piège de l'échelle
    - 6.8 🔒 Clés mutables : l'objet perdu
    - 6.9 🔒 Copie défensive : `getItems()`
    - 6.10 🔵 Choisir son implémentation : `LinkedHashMap`, `TreeMap`, `ArrayDeque`…
    - 6.11 🟣 Écrire ses propres génériques
    - 6.12 🔵 `Deque` : une file à deux bouts
7. `Optional`
    - 7.1 🟢 Pourquoi `Optional`
    - 7.2 🟢 Utiliser `Optional`
    - 7.3 🟢 Les pièges d'`Optional`
8. Le modèle FoodFast complet
9. Ce qu'il faut retenir
10. Pour aller plus loin

Objectifs :

- **AA2** — Modéliser un domaine avec des classes, des `record` et des `enum`, en appliquant l'encapsulation et des invariants.
- **AA3** — Utiliser l'héritage, les interfaces et le polymorphisme.
- **AA4** — Choisir et utiliser la bonne collection (`List`, `Set`, `Map`) et expliquer le rôle d'`equals` / `hashCode`.

Prérequis : Séance 1 (types, conditions, boucles, tableaux, méthodes `static`, `String`, Maven, JUnit).

Comment lire ce support : 🟢 indispensable · 🔵 pour approfondir · 🟣 bonus · 🔒 sécurité

À propos des exemples. Chaque exemple marqué `// Fichier : Nom.java` est compilé et exécuté avec le JDK 25. La plupart sont des *compact source files* (`void main() { … }`, vus en S1) : lancez-les avec `java Nom.java`. Un compact source file importe automatiquement tout le module `java.base` : `BigDecimal`, `List`, `Map`, `LocalDateTime`, `UUID`… sont disponibles sans `import`. Dans un projet Maven, avec des classes ordinaires, ajoutez les imports (`java.math.BigDecimal`, `java.util.*`, `java.time.LocalDateTime`…).

Le fil rouge : à la fin de cette séance, le modèle FoodFast complet existe (§8). On le construit pas à pas, dans cet ordre :

Partie	Ce qu'on construit
1. Classe et objet	une première version de `Dish`, puis `Order`
2. `enum` et `record`	`DishSize`, `OrderStatus`, `Dish` devient un `record`
3. Types riches	le prix de `Dish` passe en `BigDecimal` ; `LocalDateTime`, `UUID`
4. Héritage	`User`, `Customer`, `Courier`
5. Interfaces	`Discount`, `PercentageDiscount`, `FixedDiscount`
6. Collections	les plats d'une `Order`, les commandes de `DeliveryPlatform`
7. `Optional`	`DeliveryPlatform.findOrderById`

---

1. Classe et objet

1.1 🟢 Pourquoi des classes

Le problème. En S1, pour gérer un menu, vous n'aviez que des tableaux et des types simples. Un plat, c'est un nom, un prix et une taille : il faut donc trois tableaux « parallèles », reliés uniquement par leur

indice.

```
// S1 : un menu = des tableaux parallèles
String[] names    = {"Margherita", "Burger"};
int[] pricesCents = {1250, 990};
String[] sizes    = {"MEDIUM", "LARGE"};
```

Ce code est fragile :

- pour supprimer ou trier un plat, il faut toucher les trois tableaux, dans le même ordre ;
- rien n'empêche `pricesCents[1] = -500` ;
- un oubli ne provoque **aucune erreur** : les données deviennent incohérentes en silence.

La solution. Une **classe** regroupe, dans une même unité, les **données** d'un concept métier (ses *champs*) et les **règles** qui s'y appliquent (ses *méthodes*). Un **objet** est un exemplaire concret de cette classe : *cette* Margherita à 12,50 €.

Notion	Analogie	FoodFast
Classe	le plan, le moule	`Dish`
Objet (instance)	une pièce fabriquée avec le moule	la Margherita, le Burger
Champ	une caractéristique	`name`, `priceInCents`
Méthode	ce que l'objet sait faire	`isCheaperThan(other)`

1.2 🟢 Champs, constructeur, `this`

```
// Fichier : FirstDish.java
class Dish {
    String name;          // champ : chaque objet a le sien
    int priceInCents;     // en centimes, comme en S1 (on passera à BigDecimal en §3)

    Dish(String name, int priceInCents) {   // constructeur
        this.name = name;                    // this.name = le champ, name = le paramètre
        this.priceInCents = priceInCents;
    }

    boolean isCheaperThan(Dish other) {      // méthode d'instance
        return this.priceInCents < other.priceInCents;
    }
}

void main() {
    Dish pizza = new Dish("Margherita", 1250);
    Dish burger = new Dish("Burger", 990);
    IO.println(pizza.name + " : " + pizza.priceInCents + " centimes");
    IO.println(burger.isCheaperThan(pizza)); // true
}
```

Ce qu'il faut retenir :

- Le **constructeur** porte le nom de la classe et n'a **pas de type de retour**. Il est appelé par `new`.
- `this` désigne l'objet en cours de construction (ou celui sur lequel la méthode est appelée). `this.name = name` copie le paramètre dans le champ.
- Chaque `new` crée un **nouvel objet** sur le tas ; la variable `pizza` contient une **référence** vers lui (rappel du schéma pile/tas de S1).
- Si vous n'écrivez aucun constructeur, Java en fournit un sans paramètre. Dès que vous en écrivez un, celui-là disparaît.

> **Piège** — Oublier `this` quand le paramètre a le même nom que le champ.
>
> ```
> Dish(String name, int priceInCents) {
>     name = name;   // ❌ affecte le paramètre à lui-même : le champ reste null
> }
> ```
>
> ✅ `this.name = name;`

1.3 🟢 Encapsulation et invariants

Le problème. Avec des champs accessibles, n'importe quel code peut casser l'objet :

```
Dish pizza = new Dish("Margherita", 1250);
pizza.priceInCents = -500;   // ❌ compile, et personne ne s'en rend compte
```

La solution. Un **invariant** est une règle toujours vraie pour un objet valide : « un prix n'est jamais négatif », « un nom n'est jamais vide ». On le garantit en deux temps :

1. le **constructeur vérifie** les valeurs et **refuse** celles qui sont invalides ;
2. les champs sont `private` (inaccessibles hors de la classe) et, si possible, `final` (non réaffectables).

Ainsi, un objet invalide **ne peut pas exister** : on ne vérifie la règle qu'une fois, à la construction.

Pour refuser une valeur, on écrit `throw new IllegalArgumentException("message")`. Retenez simplement : « cette instruction arrête tout et signale que l'argument est invalide ». Les exceptions seront étudiées en détail en S3.

```java
// Fichier : ValidatedDish.java (exception attendue)
class Dish {
    private final String name;
    private final int priceInCents;

    Dish(String name, int priceInCents) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Nom vide");
        }
        if (priceInCents < 0) {
            throw new IllegalArgumentException("Prix négatif : " + priceInCents);
        }
        this.name = name;
        this.priceInCents = priceInCents;
    }

    String getName() { return name; }
    int getPriceInCents() { return priceInCents; }
}

void main() {
    Dish pizza = new Dish("Margherita", 1250);
    IO.println(pizza.getName() + " : " + pizza.getPriceInCents());
    Dish broken = new Dish("Burger", -500);   // refusé : le programme s'arrête ici
    IO.println("Jamais affiché");
}
```

Sortie :

```
Margherita : 1250
Exception in thread "main" java.lang.IllegalArgumentException: Prix négatif : -500
```

Les modificateurs d'accès :

Modificateur	Visible depuis	Usage typique
`private`	la classe seulement	**tous les champs**
(rien)	le même package	classes et méthodes internes au module
`protected`	le package et les sous-classes	constructeur d'une classe mère (§4)
`public`	partout	l'API de la classe

> 🔒 **Sécurité** — Un objet qui vérifie ses invariants à la construction ne peut pas être créé dans un état invalide : c'est la première ligne de défense contre les entrées malveillantes ou erronées (rappel S1 : valider les entrées). Vérifiez **tout** dans le constructeur : `null`, chaînes vides, bornes. Voir SEI CERT, règle *OBJ01-J. Limit accessibility of fields*.

1.4 🟢 Méthodes métier plutôt que setters

Le problème. Le réflexe « tous les champs en `private`, puis un getter et un setter pour chacun » (souvent généré par l'IDE) annule l'encapsulation : un `setPriceInCents(int)` public permet de remettre un prix négatif, sauf à revérifier la règle dans chaque setter.

La solution. Demandez-vous : **quelles opérations le métier autorise-t-il ?** On « ajoute un plat », on « confirme une commande ». On ne « modifie pas le nombre de plats » directement. Chaque méthode métier est un **point de contrôle** où l'on vérifie les règles.

```
// Fichier : OrderBusiness.java
class Order {
    private int dishCount;
    private boolean confirmed;

    void addDish(int quantity) {
        if (confirmed) {
            throw new IllegalStateException("Commande déjà confirmée");
        }
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantité invalide : " + quantity);
        }
        dishCount += quantity;
    }

    void confirm() {
        if (dishCount == 0) {
            throw new IllegalStateException("Commande vide");
        }
        confirmed = true;
    }

    int getDishCount() { return dishCount; }
    boolean isConfirmed() { return confirmed; }
}

void main() {
    Order order = new Order();
    order.addDish(2);
    order.addDish(1);
    order.confirm();
    IO.println(order.getDishCount() + " plats, confirmée : " + order.isConfirmed());
    // 3 plats, confirmée : true
}
```

`IllegalStateException` fonctionne comme `IllegalArgumentException`, mais signale que **l'objet** n'est pas dans le bon état pour l'opération (ici : commande déjà confirmée).

❌ Setters à la chaîne	✅ Méthodes métier
`setDishCount(-3)` accepté	`addDish(-3)` refusé

❌ Setters à la chaîne	✅ Méthodes métier
`setConfirmed(false)` : on « dé-confirme »	pas de méthode pour revenir en arrière
les règles sont dispersées chez les appelants	les règles sont dans la classe

Règle pratique : un **getter** si quelqu'un en a besoin ; un **setter** presque jamais.

1.5 🟢 `static` contre instance

En S1, toutes vos méthodes étaient `static` : elles ne connaissaient que leurs paramètres. Maintenant que nous avons des objets, il faut distinguer :

	Instance (sans `static`)	Classe (`static`)
Appartient à	chaque objet	la classe, en un seul exemplaire
Appel	`order.addDish(2)`	`Order.MAX_QUANTITY`, `Math.max(a, b)`
Accès à `this`	oui	**non**
Exemple	`dishCount`, `addDish`	constantes, méthodes utilitaires

```
// Fichier : StaticDemo.java
class Order {
    static final int MAX_QUANTITY = 20;   // une seule valeur, partagée
    private int dishCount;                // une valeur par commande

    void addDish(int quantity) {
        if (quantity <= 0 || quantity > MAX_QUANTITY) {
            throw new IllegalArgumentException("Quantité hors bornes : " + quantity);
        }
        dishCount += quantity;
    }

    int getDishCount() { return dishCount; }

    static boolean isValidQuantity(int quantity) {   // n'utilise aucun champ
        return quantity > 0 && quantity <= MAX_QUANTITY;
    }
}

void main() {
    Order a = new Order();
    Order b = new Order();
    a.addDish(3);
    IO.println(a.getDishCount() + " / " + b.getDishCount()); // 3 / 0
    IO.println(Order.MAX_QUANTITY);                          // 20
    IO.println(Order.isValidQuantity(25));                   // false
}
```

> **Piège** — Une méthode `static` ne peut pas utiliser un champ d'instance : quel objet ? Le compilateur refuse `static int count() { return dishCount; }` (« non-static variable cannot be referenced from a static context »). Ne « corrigez » pas en rendant le champ `static` : toutes les commandes partageraient alors le même compteur.

---

2. `enum` et `record`

2.1 🟢 `enum`

Le problème. Une taille de plat ou un statut de commande ne peut prendre qu'un petit nombre de valeurs connues. Avec une `String`, tout passe :

```
String status = "PENDNG";   // ❌ faute de frappe : compile, bug découvert en production
```

La solution. Un `enum` déclare un **ensemble fermé** de valeurs. Le compilateur les connaît et refuse tout le reste.

```
// Fichier : EnumDemo.java
enum DishSize { SMALL, MEDIUM, LARGE }

enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

void main() {
    OrderStatus status = OrderStatus.PENDING;
    // OrderStatus typo = OrderStatus.PENDNG;   // ❌ erreur de compilation

    IO.println(status);                          // PENDING
    IO.println(status == OrderStatus.PENDING);   // true : on compare avec ==

    for (DishSize size : DishSize.values()) {    // toutes les valeurs, dans l'ordre
        IO.print(size + " ");
    }
    IO.println();                                // SMALL MEDIUM LARGE

    OrderStatus parsed = OrderStatus.valueOf("CONFIRMED");   // depuis une String
    IO.println(parsed.name());                   // CONFIRMED
}
```

Ce qu'il faut retenir :

- Chaque valeur d'un `enum` est un **objet unique** : on peut comparer avec `==`.
- `values()` renvoie toutes les valeurs ; `valueOf("…")` convertit une `String` (et refuse une valeur inconnue avec une `IllegalArgumentException`).

- Un `enum` est un type à part entière : on l'utilise comme type de champ (`private OrderStatus status`) ou de paramètre.

2.2 🟢 `switch` sur un `enum`

Le `switch` en expression (S1) prend tout son sens avec un `enum` :

```
// Fichier : StatusLabel.java
enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

static String label(OrderStatus status) {
    return switch (status) {
        case PENDING     -> "En attente";
        case CONFIRMED   -> "Confirmée";
        case PREPARING   -> "En préparation";
        case IN_DELIVERY -> "En livraison";
        case DELIVERED   -> "Livrée";
        case CANCELLED   -> "Annulée";
    };
}

static boolean canBeCancelled(OrderStatus status) {
    return switch (status) {
        case PENDING, CONFIRMED -> true;                        // plusieurs valeurs
        case PREPARING, IN_DELIVERY, DELIVERED, CANCELLED -> false;
    };
}

void main() {
    IO.println(label(OrderStatus.IN_DELIVERY));        // En livraison
    IO.println(canBeCancelled(OrderStatus.PREPARING)); // false
}
```

Exhaustivité. Le `switch` ci-dessus n'a **pas de `default`**, et c'est voulu. Le compilateur vérifie que tous les cas sont traités. Si l'on ajoute `REFUNDED` à `OrderStatus`, ces deux méthodes **ne compilent plus** : le compilateur vous liste les endroits à mettre à jour.

> **Piège** — Ajouter un `default` « par sécurité ».
>
> ```
> case DELIVERED -> "Livrée";
> default -> "Inconnu";    // ❌ masque l'oubli : REFUNDED affichera « Inconnu »
> ```
>
> ✅ Sur un `enum`, traitez tous les cas, sans `default`.

2.3 🔵 `enum` avec champs et méthodes ; le piège `ordinal()`

Un `enum` peut porter des champs, un constructeur et des méthodes. Exemple : un libellé et un indicateur « statut final » associés à chaque statut. (C'est une variante : le modèle de référence garde un

`OrderStatus` simple et utilise un `switch`.)

```
// Fichier : RichEnum.java
enum OrderStatus {
    PENDING("En attente", false),
    CONFIRMED("Confirmée", false),
    PREPARING("En préparation", false),
    IN_DELIVERY("En livraison", false),
    DELIVERED("Livrée", true),
    CANCELLED("Annulée", true);

    private final String label;
    private final boolean isFinal;

    OrderStatus(String label, boolean isFinal) {   // constructeur toujours privé
        this.label = label;
        this.isFinal = isFinal;
    }

    String label() { return label; }
    boolean isFinal() { return isFinal; }
}

void main() {
    for (OrderStatus s : OrderStatus.values()) {
        IO.println(s + " → " + s.label() + (s.isFinal() ? " (final)" : ""));
    }
}
```

> **Piège** — `ordinal()` renvoie la position de la valeur dans la déclaration (`PENDING` = 0, `CONFIRMED` = 1…). Ne la stockez **jamais** (base de données, fichier) et ne fondez aucune règle métier dessus : il suffit d'insérer une valeur au milieu de l'`enum` pour décaler toutes les autres. ✅ Stockez `name()` (`"CONFIRMED"`) ou un champ dédié.

2.4 🟢 `record`

Le problème. Écrire une classe « valeur » complète à la main est long et risqué. Pour `Dish`, il faut : trois champs `private final`, un constructeur, trois accesseurs, `equals`, `hashCode` et `toString` (nous verrons en §6.6 pourquoi les deux premiers sont indispensables). Environ 40 lignes, où un seul oubli crée un bug.

La solution. Un `record` (Java 16) déclare une **valeur immuable** en une ligne. Java génère tout le reste.

```java
// Fichier : RecordDemo.java
enum DishSize { SMALL, MEDIUM, LARGE }

record Dish(String name, int priceInCents, DishSize size) {}

void main() {
    Dish pizza = new Dish("Margherita", 1250, DishSize.MEDIUM);
    Dish same  = new Dish("Margherita", 1250, DishSize.MEDIUM);

    IO.println(pizza.name());         // Margherita : accesseur, sans « get »
    IO.println(pizza);                // Dish[name=Margherita, priceInCents=1250,
size=MEDIUM]
    IO.println(pizza.equals(same));   // true : comparaison champ par champ
    IO.println(pizza == same);        // false : deux objets distincts
}
```

Ce que génère `record Dish(String name, int priceInCents, DishSize size)` :

Élément	Généré
Champs	`private final String name;` etc.
Constructeur	`Dish(String name, int priceInCents, DishSize size)`
Accesseurs	`name()`, `priceInCents()`, `size()` (pas de `get`)
`equals` / `hashCode`	sur **tous** les champs
`toString`	`Dish[name=…, priceInCents=…, size=…]`

Un `record` ne peut pas avoir d'autres champs d'instance que ses composants, et ses champs ne peuvent pas être réaffectés : il est **immuable** par construction. Il peut en revanche avoir des méthodes, des méthodes `static` et implémenter des interfaces (§5).

2.5 🟢 Le constructeur compact

Le problème. Le constructeur généré accepte n'importe quoi : `new Dish("", -500, null)`.

La solution. Le **constructeur compact** s'écrit sans liste de paramètres. Il s'exécute **avant** que Java affecte les champs : on y place les vérifications (et éventuellement des normalisations, voir §3.3).

```java
// Fichier : CompactConstructor.java (exception attendue)
enum DishSize { SMALL, MEDIUM, LARGE }

record Dish(String name, int priceInCents, DishSize size) {
    Dish {                                  // pas de parenthèses, pas de this.x = x
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Nom vide");
        }
        if (priceInCents < 0) {
            throw new IllegalArgumentException("Prix négatif : " + priceInCents);
        }
        Objects.requireNonNull(size, "size");   // refuse null
    }
}

void main() {
    IO.println(new Dish("Burger", 990, DishSize.LARGE));
    new Dish("  ", 990, DishSize.LARGE);     // IllegalArgumentException: Nom vide
}
```

`Objects.requireNonNull(x, "x")` est un raccourci : si `x` vaut `null`, il arrête le programme avec une `NullPointerException` portant le message `"x"`. C'est la manière standard de refuser `null`.

> **Piège** — Écrire `this.name = name;` dans un constructeur compact : erreur de compilation. Java fait l'affectation lui-même, à la fin du constructeur compact.

2.6 🔵 `record` ou classe ?

Question	`record`	Classe
L'objet est-il défini par ses valeurs ? (deux Margherita à 12,50 € sont « la même »)	✅ **valeur** : `Dish`, `PercentageDiscount`	
L'objet a-t-il une identité et un cycle de vie ? (une commande passe de `PENDING` à `DELIVERED`)		✅ **entité** : `Order`
L'état change-t-il ?	jamais	oui, via des méthodes métier
Héritage	ne peut pas hériter d'une classe (peut implémenter des interfaces)	oui

Dans FoodFast : `Dish`, `PercentageDiscount`, `FixedDiscount` sont des records. `Order` est une classe (son statut et ses plats changent). `User` est une classe abstraite, car elle sert de base à `Customer` et `Courier` (§4).

> 🔒 **Sécurité** — Un record est **immuable superficiellement** : ses champs ne changent pas, mais si un champ est une `List` modifiable, la liste, elle, peut changer. Pour un record qui contient une collection, faites une copie dans le constructeur compact : `items = List.copyOf(items);` (voir §6.9).

---

3. Types riches

3.1 🟢 Le problème du `double`

```
// Fichier : DoubleProblem.java
void main() {
    double a = 0.1 + 0.2;
    IO.println(a);              // 0.30000000000000004
    IO.println(a == 0.3);       // false

    double total = 0.0;
    for (int i = 0; i < 100; i++) {
        total += 0.01;          // ajouter 1 centime, 100 fois
    }
    IO.println(total);          // 1.0000000000000007

    // La solution de S1 : des centimes en int. Exacte pour additionner...
    int totalCents = 0;
    for (int i = 0; i < 100; i++) {
        totalCents += 1;
    }
    IO.println(totalCents);     // 100
    // ...mais 10 % de remise sur 1245 centimes ? 124,5 centimes : quel arrondi ?
    IO.println(1245 * 10 / 100);   // 124 : troncature silencieuse
}
```

Pourquoi. Un `double` est stocké en **binaire** (norme IEEE 754). Comme 1/3 n'a pas d'écriture décimale finie, 0,1 n'a pas d'écriture binaire finie : la machine en stocke une approximation. Ce n'est pas un bug de Java : Python, JavaScript ou C donnent le même résultat.

En S1, vous avez contourné le problème avec des **centimes en `int`**. C'est exact pour additionner, mais :

- les pourcentages et les divisions **tronquent** sans prévenir ;
- l'affichage (`1245` → `12,45 €`) est à recoder partout ;
- le débordement d'entier reste possible (S1 : `Math.addExact`).

`BigDecimal` généralise l'idée des centimes : un nombre **décimal exact**, avec un **arrondi explicite** quand il en faut un.

3.2 🟢 `BigDecimal` : créer et calculer

Documentation : `java.math.BigDecimal`

```
// Fichier : BigDecimalBasics.java
void main() {
    BigDecimal price = new BigDecimal("12.50");      // ✅ depuis une String : exact
    BigDecimal wrong = new BigDecimal(0.1);          // ❌ hérite de l'erreur du double
    IO.println(wrong);   // 0.1000000000000000055511151231257827021181583404541015625

    BigDecimal two = BigDecimal.valueOf(2);          // ✅ depuis un entier
    BigDecimal total = price.multiply(two);
    IO.println(total);                               // 25.00

    total.add(new BigDecimal("2.90"));               // ❌ résultat ignoré
    IO.println(total);                               // 25.00 : inchangé !

    total = total.add(new BigDecimal("2.90"));       // ✅ on récupère le résultat
    IO.println(total);                               // 27.90

    BigDecimal sum = new BigDecimal("0.1").add(new BigDecimal("0.2"));
    IO.println(sum);                                 // 0.3
    IO.println(sum.compareTo(new BigDecimal("0.3")) == 0);   // true
}
```

Les règles :

1. **Construire depuis une `String`** : `new BigDecimal("12.50")`. Pour un entier : `BigDecimal.valueOf(2)`. Jamais `new BigDecimal(double)`.
2. **`BigDecimal` est immuable**, comme `String` : `add`, `subtract`, `multiply`, `divide` renvoient un **nouvel** objet. Il faut récupérer le résultat.
3. **Comparer avec `compareTo`** (§3.3).
4. Les constantes utiles : `BigDecimal.ZERO`, `BigDecimal.ONE`, `BigDecimal.TEN`.

Opération	Méthode
a + b	`a.add(b)`
a – b	`a.subtract(b)`
a × b	`a.multiply(b)`
a ÷ b	`a.divide(b, 2, RoundingMode.HALF_UP)` (voir §3.4)
signe	`a.signum()` : –1, 0 ou 1
maximum	`a.max(b)`

3.3 🟢 `equals` contre `compareTo`, et la version finale de `Dish`

Un `BigDecimal` a une **valeur** et une **échelle** (le nombre de chiffres après la virgule) : `12.5` a l'échelle 1, `12.50` l'échelle 2.

- `equals` compare la valeur **et** l'échelle : `12.5` ≠ `12.50`.
- `compareTo` compare la valeur seulement : il renvoie un nombre négatif, zéro ou positif.

```
// Fichier : BigDecimalEquality.java
void main() {
    BigDecimal a = new BigDecimal("12.5");
    BigDecimal b = new BigDecimal("12.50");

    IO.println(a.equals(b));             // false : échelles 1 et 2
    IO.println(a.compareTo(b) == 0);     // true  : même valeur
    IO.println(a.hashCode() == b.hashCode());   // false

    BigDecimal c = a.setScale(2, RoundingMode.HALF_UP);
    IO.println(c + " " + c.equals(b));   // 12.50 true
}
```

Conséquence pour `Dish`. Un record compare ses champs avec `equals`. Si le prix est un `BigDecimal` brut, `Dish("Pizza", 12.5)` et `Dish("Pizza", 12.50)` sont **deux plats différents**, avec deux `hashCode` différents. Dans une `Map<Dish, Integer>`, ils deviendraient deux lignes distinctes (démonstration en §6.7).

La solution : normaliser dans le constructeur compact. Dans un constructeur compact, on peut **réaffecter le paramètre** avant que Java remplisse le champ. On fixe l'échelle à 2 décimales : tous les prix sont alors comparables avec `equals`.

Voici la **version finale** de `Dish`, celle du modèle de référence :

```
// Fichier : FinalDish.java
enum DishSize { SMALL, MEDIUM, LARGE }

/** Valeur immuable : un plat du menu. */
record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        if (name.isBlank()) throw new IllegalArgumentException("Nom vide");
        if (price.signum() < 0) throw new IllegalArgumentException("Prix négatif : " +
price);
        price = price.setScale(2, RoundingMode.HALF_UP); // 12.5 et 12.50 deviennent égaux
    }
}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.5"), DishSize.MEDIUM);
    Dish same  = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
    IO.println(pizza.price());         // 12.50
    IO.println(pizza.equals(same));    // true
    IO.println(pizza.hashCode() == same.hashCode());   // true
}
```

Remarquez l'ordre : on vérifie `null` **avant** d'appeler `price.signum()`. Dans l'autre ordre, un prix `null` provoquerait une `NullPointerException` sans message clair au lieu du refus prévu.

> **Piège —** `if (total.equals(BigDecimal.ZERO))` : faux si `total` vaut `0.00`. ✅ `total.signum() == 0` ou `total.compareTo(BigDecimal.ZERO) == 0`.

3.4 🔵 Division et arrondis

Une division peut avoir un développement décimal infini (10 ÷ 3). Sans précision demandée, `BigDecimal` refuse :

```
// Fichier : DivideWithoutScale.java (exception attendue)
void main() {
    BigDecimal total = new BigDecimal("10.00");
    BigDecimal share = total.divide(BigDecimal.valueOf(3));
    // ArithmeticException: Non-terminating decimal expansion; no exact representable
decimal result.
    IO.println(share);
}
```

✅ Toujours préciser l'échelle et le mode d'arrondi :

```java
// Fichier : Rounding.java
void main() {
    BigDecimal total = new BigDecimal("10.00");
    IO.println(total.divide(BigDecimal.valueOf(3), 2, RoundingMode.HALF_UP));   // 3.33

    BigDecimal[] values = { new BigDecimal("2.345"), new BigDecimal("2.355") };
    for (BigDecimal v : values) {
        IO.println(v + " → HALF_UP " + v.setScale(2, RoundingMode.HALF_UP)
                + " / HALF_EVEN " + v.setScale(2, RoundingMode.HALF_EVEN));
    }
    // 2.345 → HALF_UP 2.35 / HALF_EVEN 2.34
    // 2.355 → HALF_UP 2.36 / HALF_EVEN 2.36

    NumberFormat euros = NumberFormat.getCurrencyInstance(Locale.FRANCE);
    IO.println(euros.format(new BigDecimal("1234.5")));   // 1 234,50 € (espaces
insécables)
}
```

Mode	Règle	Usage
`HALF_UP`	0,5 arrondi vers le haut (l'arrondi « de l'école »)	prix affichés, choix du modèle FoodFast
`HALF_EVEN`	0,5 arrondi vers le chiffre **pair** (« arrondi bancaire »)	comptabilité, calculs en série : les arrondis ne s'accumulent pas dans un sens
`DOWN`	troncature	rarement souhaité

Pour afficher un montant, utilisez `NumberFormat.getCurrencyInstance(Locale.FRANCE)` ou `toPlainString()`. Ne repassez jamais par un `double`.

3.5 🟢 `LocalDateTime`

L'API `java.time` (Java 8) remplace `Date` et `Calendar`, qu'il ne faut plus utiliser dans du code neuf. Ses classes sont **immuables**, comme `BigDecimal` : `plusMinutes` renvoie un nouvel objet.

Classe	Représente	Exemple
`LocalDate`	une date	anniversaire du client
`LocalTime`	une heure	heure d'ouverture du restaurant
`LocalDateTime`	date + heure, sans fuseau	`Order.createdAt`
`Duration`	une durée en secondes/minutes	temps de livraison

```java
// Fichier : DatesDemo.java
void main() {
    LocalDateTime createdAt = LocalDateTime.of(2026, 10, 24, 19, 30);   // date fixe pour
l'exemple
    LocalDateTime deadline = createdAt.plusMinutes(45);
    LocalDateTime deliveredAt = LocalDateTime.of(2026, 10, 24, 20, 20);

    IO.println(deadline);                          // 2026-10-24T20:15
    IO.println(deliveredAt.isAfter(deadline));     // true : en retard
    IO.println(Duration.between(createdAt, deliveredAt).toMinutes());   // 50

    DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    IO.println(createdAt.format(fr));              // 24/10/2026 19:30

    LocalDateTime now = LocalDateTime.now();       // l'instant présent (varie à chaque
exécution)
    IO.println(now.isAfter(createdAt.minusYears(100)));   // true
}
```

🔵 **Fuseaux horaires** — `LocalDateTime` n'a pas de fuseau. Pour un instant absolu (journalisation, serveurs dans plusieurs pays), utilisez `Instant` ou `ZonedDateTime`.

3.6 🟢 `UUID`

Un `UUID` (*Universally Unique Identifier*) est un identifiant de 128 bits, généré sans compteur central. `UUID.randomUUID()` produit un UUID de version 4 : 122 bits aléatoires issus d'un générateur cryptographique (`SecureRandom`).

```java
// Fichier : UuidDemo.java
void main() {
    UUID uuid = UUID.randomUUID();
    String orderId = "order-" + uuid;     // c'est ainsi que Order construit son id
    IO.println(orderId);                  // ex. order-3f2b8c1e-9a4d-4c1b-8e2f-
0a6d5b7c9e11
    IO.println(uuid.version());           // 4
    IO.println(UUID.randomUUID().equals(uuid));   // false (en pratique, jamais égal)
}
```

🔒 **Sécurité** — Avec des identifiants séquentiels (`order-41`, `order-42`), un utilisateur peut deviner les commandes des autres en changeant le numéro dans l'URL. C'est une **IDOR** (*Insecure Direct Object Reference*), classée dans **OWASP A01 : Broken Access Control**. Un UUID aléatoire rend l'identifiant impossible à deviner, mais **ce n'est pas un contrôle d'accès** : un identifiant fuit (lien partagé, logs, historique). Le serveur doit **toujours** vérifier que la commande demandée appartient à l'utilisateur connecté.

3.7 🔵 `Instant`, `Duration` et `Clock` : le temps des machines

Pourquoi ? `LocalDateTime` décrit une date « de calendrier » (19h30 le 24 octobre), sans fuseau. Pour un serveur, on a souvent besoin d'autre chose : un **instant absolu** (« ce code promo expire dans 15 minutes », « cette session expire à tel instant »), identique partout sur la planète. Et un second problème apparaît vite : **comment tester** un code qui dépend de l'heure actuelle, sans attendre 15 minutes ?

Comment ?

Classe	Représente	Exemple
`Instant`	un point sur la ligne du temps (UTC), à la nanoseconde	`Instant.now()`, `instant.toEpochMilli()`
`Duration`	une durée en secondes et nanosecondes	`Duration.ofMinutes(15)`, `Duration.ofSeconds(90)`
`Clock`	**la source** de l'heure actuelle	`Clock.systemUTC()`, `Clock.fixed(...)`

`Instant.now()` interroge l'horloge du système : impossible à contrôler dans un test. La solution est d'**injecter l'horloge** : la classe reçoit un `Clock` dans son constructeur et appelle `Instant.now(clock)`. En production, on lui passe `Clock.systemUTC()` ; dans les tests, une horloge **figée** (`Clock.fixed`) ou **décalée** (`Clock.offset`).

Exemple concret : un code promo FoodFast valable 15 minutes.

```java
// Fichier : PromoClock.java
/** Un code promo valable pendant une durée donnée à partir de sa création. */
class TimedPromo {
    private final String code;
    private final Instant expiresAt;
    private final Clock clock;

    TimedPromo(String code, Duration validity, Clock clock) {
        this.code = code;
        this.clock = clock;
        this.expiresAt = Instant.now(clock).plus(validity);
    }

    boolean isValid() {
        return Instant.now(clock).isBefore(expiresAt);
    }

    /** Temps restant, jamais négatif. */
    Duration remaining() {
        Duration d = Duration.between(Instant.now(clock), expiresAt);
        return d.isNegative() ? Duration.ZERO : d;
    }
}

void main() {
    Instant start = Instant.parse("2026-10-24T18:00:00Z");
    Clock t0 = Clock.fixed(start, ZoneOffset.UTC);
    TimedPromo promo = new TimedPromo("FLASH15", Duration.ofMinutes(15), t0);
    IO.println(promo.isValid());                    // true
    IO.println(promo.remaining().toMinutes());      // 15

    // Simuler le passage du temps, sans attendre : la même promo, vue 20 minutes plus
tard
    Clock later = Clock.offset(t0, Duration.ofMinutes(20));
    TimedPromo seenLater = new TimedPromo("FLASH15", Duration.ofMinutes(15), t0);
    IO.println(new TimedPromo("X", Duration.ofMinutes(15), later).remaining());  // PT15M

IO.println(start.plus(Duration.ofMinutes(20)).isBefore(start.plus(Duration.ofMinutes(15))));
// false

    IO.println(start.toEpochMilli());               // 1792864800000 : millisecondes
depuis 1970
    IO.println(Duration.ofSeconds(90));             // PT1M30S (format ISO-8601)
}
```

Le dernier bloc montre une limite de cet exemple : l'horloge est fixée **à la construction**. Pour simuler le passage du temps sur **le même objet**, on utilise une horloge que le test peut avancer. `Clock` est une classe abstraite : on peut en écrire une petite sous-classe pour les tests.

```java
// Fichier : MutableClockDemo.java
/** Horloge de test : figée, mais qu'on peut avancer à la main. */
class MutableClock extends Clock {
    private Instant now;
    MutableClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }

    @Override public Instant instant() { return now; }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
}

class TimedPromo {
    private final Instant expiresAt;
    private final Clock clock;
    TimedPromo(Duration validity, Clock clock) {
        this.clock = clock;
        this.expiresAt = Instant.now(clock).plus(validity);
    }
    boolean isValid() { return Instant.now(clock).isBefore(expiresAt); }
}

void main() {
    MutableClock clock = new MutableClock(Instant.parse("2026-10-24T18:00:00Z"));
    TimedPromo promo = new TimedPromo(Duration.ofMinutes(15), clock);
    IO.println(promo.isValid());          // true
    clock.advance(Duration.ofMinutes(14));
    IO.println(promo.isValid());          // true
    clock.advance(Duration.ofMinutes(1));
    IO.println(promo.isValid());          // false : 15 minutes écoulées
}
```

C'est aussi un bel exemple d'**héritage** (§4) : `MutableClock` redéfinit les méthodes abstraites de `Clock`, et tout le code qui attend un `Clock` l'accepte (polymorphisme).

Piège.

❌ Appeler `Instant.now()` partout dans le code métier : le comportement dépend de l'heure réelle, et les tests deviennent lents (`Thread.sleep`) ou aléatoires.

✅ Injecter un `Clock` et appeler `Instant.now(clock)`. Le test contrôle le temps.

❌ Stocker une expiration en `LocalDateTime` sur un serveur : au changement d'heure d'hiver, la même heure locale existe deux fois.

✅ Un `Instant` (ou un nombre de millisecondes depuis 1970, `toEpochMilli()`) pour tout ce qui est « machine ».

---

4. Héritage

4.1 🟢 Pourquoi l'héritage

Le problème. Un client et un livreur ont tous les deux un identifiant, un nom et un email. Sans héritage, on copie-colle les trois champs, leurs getters et leurs validations dans `Customer` et dans `Courier`. Toute correction doit être faite deux fois.

La solution. On écrit le commun une fois, dans une classe mère `User`, et les classes filles **héritent** de ses champs et méthodes.

```
            Object          ← racine de toutes les classes Java
              │
        User (abstract)     ← id, name, email, role()
          ┌───┴────┐
    Customer    Courier     ← Customer ajoute address ; Courier n'ajoute rien
```

La relation d'héritage se lit « **est un** » : un `Customer` **est un** `User`. Partout où l'on attend un `User`, on peut passer un `Customer`.

4.2 🟢 `extends`, `super(...)`, `protected`

```java
// Fichier : Users.java
/** Héritage : ce que partagent tous les utilisateurs de la plateforme. */
abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;

    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public abstract String role();
}

class Customer extends User {
    private final String address;
    Customer(String name, String email, String address) {
        super(name, email);                         // 1. construit la partie User
        this.address = Objects.requireNonNull(address);   // 2. puis la partie Customer
    }
    public String getAddress() { return address; }
    @Override public String role() { return "CLIENT"; }
}

class Courier extends User {
    Courier(String name, String email) { super(name, email); }
    @Override public String role() { return "LIVREUR"; }
}

void main() {
    Customer alice = new Customer("Alice", "alice@example.com", "10 rue Haute");
    Courier bob = new Courier("Bob", "bob@example.com");

    IO.println(alice.getName() + " habite " + alice.getAddress());   // hérité + propre
    User[] users = { alice, bob };          // un Customer EST un User
    for (User u : users) {
        IO.println(u.getName() + " : " + u.role());
    }
    // Alice : CLIENT
    // Bob : LIVREUR
}
```

Ce qu'il faut retenir :

- `class Customer extends User` : `Customer` hérite des champs et des méthodes de `User`.
- `super(name, email)` appelle le constructeur de la classe mère. Il construit la « partie `User` » de l'objet. Si vous ne l'écrivez pas, Java appelle `super()` sans argument, qui n'existe pas ici : erreur de compilation.

- Les champs `private` de `User` **existent** dans un `Customer`, mais `Customer` n'y accède que par les méthodes publiques (`getName()`). L'encapsulation vaut aussi entre mère et fille.
- `protected` sur le constructeur : seules les sous-classes (et le package) peuvent l'appeler.

> **Piège** — Mettre les champs de la classe mère en `protected` « pour que les filles y accèdent ». Les sous-classes peuvent alors casser les invariants de la mère. ✅ Champs `private`, accès par méthodes.

4.3 🟢 `@Override`, `abstract` et polymorphisme

- Une méthode `abstract` n'a pas de corps : `public abstract String role();`. Chaque sous-classe concrète **doit** la fournir.
- Une classe qui contient une méthode abstraite est elle-même `abstract` : on ne peut pas l'instancier. `new User(...)` ne compile pas, et c'est logique : un « utilisateur » qui n'est ni client ni livreur n'existe pas dans FoodFast.
- **Redéfinir** (*override*) une méthode, c'est en fournir une nouvelle version dans la sous-classe, avec la même signature.
- `@Override` demande au compilateur de vérifier qu'on redéfinit bien une méthode existante. Sans lui, une faute de frappe crée silencieusement une nouvelle méthode.

```
// Fichier : OverrideTypo.java
class Product {
    String describe() { return "produit"; }
}

class Drink extends Product {
    String descibe() { return "boisson"; }   // ❌ faute de frappe, sans @Override : nouvelle méthode
}

void main() {
    Product p = new Drink();
    IO.println(p.describe());   // « produit » : la redéfinition n'a jamais eu lieu
    // ✅ Avec @Override sur descibe(), le compilateur aurait refusé :
    //    « method does not override or implement a method from a supertype »
}
```

Polymorphisme. Dans `User u = alice; u.role();`, la variable est de type `User`, mais l'objet est un `Customer` : c'est la méthode **de l'objet** qui s'exécute (« CLIENT »). Ce choix se fait à l'exécution. Le code qui manipule des `User` n'a pas besoin de savoir s'il s'agit d'un client ou d'un livreur.

> 🔵 **`final`** — Une méthode `final` ne peut pas être redéfinie ; une classe `final` ne peut pas être étendue (`String` est `final`). Utile pour empêcher une sous-classe de contourner une règle de sécurité (CERT *OBJ00-J*).

4.4 🟢 `Object`, la racine

Toute classe qui n'écrit pas `extends` hérite implicitement de `java.lang.Object`. C'est de là que viennent trois méthodes que tout objet possède :

Méthode	Par défaut (dans `Object`)	À redéfinir quand
`toString()`	`Customer@1b6d3586` : nom de la classe + `@` + `hashCode` d'identité en hexadécimal	toujours utile pour le débogage
`equals(Object)`	`this == other` : même objet en mémoire	égalité **de valeur** voulue (§6.6)
`hashCode()`	un nombre lié à l'identité de l'objet	**en même temps** qu'`equals`

```
// Fichier : ObjectRoot.java
class Courier {
    private final String name;
    Courier(String name) { this.name = name; }
}

class Customer {
    private final String name;
    private final String email;
    Customer(String name, String email) { this.name = name; this.email = email; }

    @Override
    public String toString() {
        return "Customer[name=" + name + "]";   // 🔒 l'email n'est pas affiché
    }
}

void main() {
    IO.println(new Courier("Bob"));                       // …Courier@35fc6dc4 : illisible
    IO.println(new Customer("Alice", "alice@example.com"));  // Customer[name=Alice]
}
```

Attention : la valeur après `@` n'est **pas** une adresse mémoire (le ramasse-miettes déplace les objets), c'est le `hashCode` d'identité.

> 🔒 **Sécurité** — `toString()` finit dans les logs, les messages d'erreur, la console de débogage. N'y mettez **jamais** de mot de passe, de numéro de carte, de jeton. Attention : le `toString()` généré d'un `record` affiche **tous** les composants. Pour un record qui contient une donnée sensible, redéfinissez `toString()`.

4.5 🟢 Composition plutôt qu'héritage

Une commande appartient à un client. Faut-il écrire `class Order extends Customer` pour récupérer l'adresse de livraison ?

```
// ❌ « une commande EST un client » ? Non.
class Order extends Customer {
    // hérite de address, mais aussi de email, role(), getId()...
    // order.role() renverrait "CLIENT" : absurde
}
```

```
// ✅ une commande A un client : c'est un champ
class Order {
    private final Customer customer;
    // ...
    String deliveryAddress() { return customer.getAddress(); }
}
```

Le test : lisez la phrase « X **est un** Y ». Si elle est fausse ou bancale, utilisez un champ (« X **a un** Y ») : c'est la **composition**.

	Héritage (`extends`)	**Composition (champ)**
Relation	« est un »	« a un »
Couplage	fort : toute modification de la mère touche les filles	faible : on n'utilise que l'API publique
Nombre	**une seule** classe mère en Java	autant de champs que nécessaire
Changeable à l'exécution	non	oui (on peut changer l'objet référencé)

Règle : **en cas de doute, composition.** L'héritage est réservé aux vraies hiérarchies stables (`User` → `Customer` , `Courier`).

> **Piège** — `class Courier extends Customer` « pour réutiliser le nom et l'email ». Un livreur n'est pas un client : il hériterait d'une adresse de livraison qui n'a pas de sens pour lui. ✅ Les deux héritent de `User` .

4.6 🟣 Valider avant `super(...)` (JEP 513)

Jusqu'à Java 24, `super(...)` devait être la **première instruction** du constructeur. On ne pouvait donc pas valider les arguments avant de construire la classe mère. Depuis **Java 25** (JEP 513, *Flexible Constructor Bodies*, finale), on peut écrire du code **avant** `super(...)` , à condition de ne pas utiliser `this` (sauf pour affecter des champs).

Intérêt 1 : refuser tôt. Pourquoi construire la partie `User` (et générer un UUID) si l'adresse est invalide ?

Intérêt 2 : initialiser un champ avant que la mère ne l'utilise. Si le constructeur de la mère appelle une méthode redéfinie, celle-ci s'exécute avant que les champs de la fille soient remplis. C'est un piège classique, que JEP 513 permet enfin de corriger.

```
// Fichier : EarlyValidation.java
abstract class User {
    private final String name;
    private final String email;

    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
        IO.println("Nouvel utilisateur : " + describe());   // appelle la version de la
fille
    }
    public String getName() { return name; }
    public abstract String describe();
}

class Customer extends User {
    private final String address;

    Customer(String name, String email, String address) {
        if (address == null || address.isBlank()) {          // ✅ validation avant super
            throw new IllegalArgumentException("Adresse vide");
        }
        this.address = address.strip();   // ✅ champ affecté AVANT super(...)
        super(name, email);
    }
    @Override public String describe() { return "client livré au " + address; }
}

void main() {
    new Customer("Alice", "alice@example.com", "  10 rue Haute ");
    // Nouvel utilisateur : client livré au 10 rue Haute
    // Avec l'ordre classique (super d'abord), on aurait lu : « client livré au null »
}
```

4.7 🔵 Les limites de l'héritage

- **Une seule classe mère** par classe. Pour combiner plusieurs capacités, on utilise des interfaces (§5).
- Une sous-classe dépend des **détails internes** de sa mère : modifier la mère peut casser les filles (problème de la « classe de base fragile »).
- Un `record` ne peut pas hériter d'une classe (il hérite implicitement de `java.lang.Record`), mais il peut implémenter des interfaces.
- Les `enum` n'héritent pas non plus d'une classe (ils héritent de `java.lang.Enum`).

4.8 🟢 Le patron de méthode : une règle que les sous-classes ne peuvent pas contourner

Pourquoi ? Plusieurs sous-classes partagent le **même déroulé** (vérifier, puis agir, puis tracer) mais diffèrent sur **une étape**. Si chaque sous-classe réécrit tout le déroulé, l'une d'elles finira par oublier la vérification.

Comment ? La classe abstraite écrit le déroulé dans une méthode `final` (aucune sous-classe ne peut la redéfinir). Cette méthode appelle une méthode `abstract` : c'est le seul « trou » que les sous-classes remplissent. On appelle cela le **patron de méthode** (*template method*).

Exemple concret : les moyens de paiement de FoodFast. Quel que soit le moyen, un montant doit être strictement positif.

```java
// Fichier : TemplateMethod.java
abstract class PaymentMethod {

    /** Le déroulé commun, identique pour tous les moyens de paiement. */
    public final String pay(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) {            // 1. vérifier
            throw new IllegalArgumentException("Montant invalide : " + amount);
        }
        String receipt = doPay(amount);                          // 2. l'étape variable
        return "[" + name() + "] " + receipt;                    // 3. tracer
    }

    /** L'étape propre à chaque moyen de paiement. */
    protected abstract String doPay(BigDecimal amount);

    protected abstract String name();
}

class CardPayment extends PaymentMethod {
    @Override protected String doPay(BigDecimal amount) { return "carte débitée de " +
amount + " €"; }
    @Override protected String name() { return "CB"; }
}

class VoucherPayment extends PaymentMethod {
    @Override protected String doPay(BigDecimal amount) { return "titre-restaurant de " +
amount + " €"; }
    @Override protected String name() { return "TR"; }
}

void main() {
    List<PaymentMethod> methods = List.of(new CardPayment(), new VoucherPayment());
    for (PaymentMethod m : methods) {
        IO.println(m.pay(new BigDecimal("12.50")));
    }
    // [CB] carte débitée de 12.50 €
    // [TR] titre-restaurant de 12.50 €
    try {
        new CardPayment().pay(new BigDecimal("-5"));
    } catch (IllegalArgumentException e) {
        IO.println(e.getMessage());          // Montant invalide : -5
    }
}
```

Ce qu'il faut remarquer :

- `pay` est `final` : une sous-classe ne peut pas la redéfinir, donc **ne peut pas sauter la vérification**.
- `doPay` est `abstract` et `protected` : chaque sous-classe la fournit, mais le code extérieur appelle `pay`, jamais `doPay` directement.
- Le code appelant ne connaît que `PaymentMethod` : c'est du polymorphisme (§4.3).

Piège.

❌ Laisser `pay` redéfinissable : une sous-classe `FreePayment` réécrit `pay` sans la vérification, et un montant négatif passe.

✅ `final` sur la méthode qui porte la règle, `abstract` sur l'étape qui varie.

> 🔒 **Sécurité** — Placer les contrôles dans une méthode `final` de la classe mère garantit qu'**aucune** sous-classe, même écrite plus tard par quelqu'un d'autre, ne les oublie. C'est la règle CERT *MET03-J* : les méthodes qui effectuent un contrôle de sécurité doivent être `private` ou `final`.

---

5. Interfaces et polymorphisme

5.1 🟢 Le problème : une remise par type

FoodFast propose plusieurs remises : un pourcentage (–10 %), un montant fixe (–5 €), et demain une remise « première commande » ou « happy hour ». Première idée :

```
// ❌ chaque nouveau type de remise oblige à modifier cette méthode
static BigDecimal applyDiscount(String type, int value, BigDecimal total) {
    if (type.equals("PERCENT")) {
        return total.multiply(BigDecimal.valueOf(100 - value))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    } else if (type.equals("FIXED")) {
        return total.subtract(BigDecimal.valueOf(value)).max(BigDecimal.ZERO);
    }
    return total;   // type inconnu (faute de frappe ?) : aucune remise, en silence
}
```

Problèmes : un `String` pour le type (fautes de frappe), un `int value` qui change de sens selon le type, une méthode qui grossit à chaque nouvelle remise, et un type inconnu ignoré sans erreur.

Ce qu'on veut : « **une remise sait s'appliquer à un total** », quelle qu'elle soit.

5.2 🟢 L'interface `Discount` et ses implémentations

Une **interface** déclare un **contrat** : des signatures de méthodes, sans état (pas de champ d'instance). Une classe ou un record qui écrit `implements Discount` s'engage à fournir ces méthodes.

```java
// Fichier : Discounts.java
/** Interface + polymorphisme. */
interface Discount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

void main() {
    BigDecimal total = new BigDecimal("42.00");
    Discount tenPercent = new PercentageDiscount(10);
    Discount fiveEuros = new FixedDiscount(new BigDecimal("5.00"));

    IO.println(tenPercent.apply(total));                          // 37.80
    IO.println(fiveEuros.apply(total));                           // 37.00
    IO.println(fiveEuros.apply(new BigDecimal("3.00")));          // 0 (BigDecimal.ZERO)
: jamais négatif
}
```

Ce qu'il faut retenir :

- Les méthodes d'une interface sont implicitement `public abstract`. L'implémentation doit donc être **`public`** (sinon : erreur de compilation, « attempting to assign weaker access privileges »).
- Un record peut implémenter une interface : c'est la combinaison idéale pour des remises qui sont de simples valeurs.
- `max(BigDecimal.ZERO)` garantit un invariant métier : une remise ne rend jamais le total négatif.

> 🔵 **Validation** — Le modèle de référence garde `PercentageDiscount` minimal. En production, on ajouterait un constructeur compact : `if (percent < 0 || percent > 100) throw new IllegalArgumentException(...)`. Sans lui, `new PercentageDiscount(150)` produirait un total négatif.

5.3 🟢 Polymorphisme

Le **polymorphisme** (« plusieurs formes ») : une variable d'un type interface peut désigner des objets de classes différentes, et l'appel `d.apply(total)` exécute la version de l'objet réel.

```java
// Fichier : Polymorphism.java
interface Discount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

/** Nouvelle remise : une nouvelle classe, et rien d'autre à modifier. */
record FirstOrderDiscount() implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.compareTo(new BigDecimal("20.00")) >= 0
                ? total.subtract(new BigDecimal("8.00"))
                : total;
    }
}

/** Ce code ne connaît que l'interface : il fonctionne avec toutes les remises. */
static BigDecimal bestPrice(BigDecimal total, Discount[] discounts) {
    BigDecimal best = total;
    for (Discount d : discounts) {
        best = best.min(d.apply(total));
    }
    return best;
}

void main() {
    Discount[] promos = {
        new PercentageDiscount(10),
        new FixedDiscount(new BigDecimal("5.00")),
        new FirstOrderDiscount()
    };
    BigDecimal total = new BigDecimal("42.00");
    for (Discount d : promos) {
        IO.println(d + " → " + d.apply(total));
    }
    IO.println("Meilleur prix : " + bestPrice(total, promos));   // 34.00
}
```

`bestPrice` n'a **pas changé** quand on a ajouté `FirstOrderDiscount`. C'est le bénéfice principal : le code appelant dépend du **contrat**, pas des implémentations.

> 🔗 **Lien avec la S3** — `Discount` n'a qu'une seule méthode abstraite. En S3, vous verrez qu'une telle interface peut être implémentée en une ligne avec une lambda.

5.4 🟢 Interface ou classe abstraite ?

	Interface	Classe abstraite
Représente	une **capacité** (« sait appliquer une remise »)	une **famille** (« est un utilisateur »)
Champs d'instance	non (constantes `static final` seulement)	oui
Constructeur	non	oui
Méthodes avec du code	`default` et `static` (§5.5)	oui
Combien par classe	**plusieurs** (`implements A, B`)	**une seule** (`extends`)
Implémentable par un `record`	oui	non
FoodFast	`Discount`	`User`

Règle : par défaut, une interface. Une classe abstraite quand les sous-classes partagent de l'**état** (des champs) et du code qui l'utilise. `User` partage `id`, `name`, `email` : classe abstraite. `Discount` ne partage rien : interface.

5.5 🔵 Méthodes `default` et `static` dans une interface

Depuis Java 8, une interface peut contenir :

- des méthodes `default` : une implémentation par défaut, héritée par toutes les classes qui implémentent l'interface (elles peuvent la redéfinir) ;
- des méthodes `static` : des utilitaires liés à l'interface, appelés par `Discount.none()`.

```java
// Fichier : DefaultMethods.java
interface Discount {
    BigDecimal apply(BigDecimal total);

    /** Combien le client économise : calculé à partir de apply, pour toutes les remises.
*/
    default BigDecimal savings(BigDecimal total) {
        return total.subtract(apply(total));
    }

    /** Une remise neutre. */
    static Discount none() {
        return new FixedDiscount(BigDecimal.ZERO);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

void main() {
    BigDecimal total = new BigDecimal("42.00");
    Discount d = new FixedDiscount(new BigDecimal("5.00"));
    IO.println(d.savings(total));                  // 5.00
    IO.println(Discount.none().savings(total));    // 0.00
}
```

Les méthodes `default` ont été introduites pour **faire évoluer une interface sans casser ses implémentations** : `List.getFirst()` (Java 21) est une méthode `default`, ajoutée sans modifier les milliers de classes qui implémentent `List`. Une méthode `default` ne peut pas accéder à des champs (l'interface n'en a pas) : elle s'appuie sur les méthodes abstraites.

5.6 🟢 `instanceof` avec motif : tester le type et nommer la variable

Pourquoi ? Une variable de type interface (ou classe mère) peut désigner des objets de types différents. Parfois, il faut savoir **lequel** pour agir. Avant Java 16, on testait le type, puis on faisait une conversion (*cast*) à la main, avec le risque de se tromper de type entre les deux.

Comment ? `x instanceof Type nom` fait trois choses à la fois : il teste que `x` est un `Type` (et qu'il n'est pas `null`), il le convertit, et il le range dans la variable `nom`, utilisable là où le test est vrai.

```java
// Fichier : InstanceofPattern.java
interface Discount { BigDecimal apply(BigDecimal total); }

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

static String label(Discount d) {
    if (d instanceof PercentageDiscount p) {          // teste ET nomme
        return "-" + p.percent() + " %";
    }
    if (d instanceof FixedDiscount f) {
        return "-" + f.amount() + " €";
    }
    return "remise";
}

void main() {
    IO.println(label(new PercentageDiscount(10)));                  // -10 %
    IO.println(label(new FixedDiscount(new BigDecimal("5.00"))));   // -5.00 €
    IO.println(null instanceof PercentageDiscount);                 // false : jamais de
NPE
}
```

Vous l'avez déjà croisé dans `equals` (§6.6) : `if (!(o instanceof PlainDish other)) return false;`.

Piège.

❌ L'ancienne forme, avec un cast séparé qu'on peut se tromper d'écrire :

```java
if (d instanceof PercentageDiscount) {
    FixedDiscount f = (FixedDiscount) d;   // compile, mais ClassCastException à
l'exécution
}
```

✅ La forme avec motif : le test et la conversion ne peuvent pas diverger.

❌ Enchaîner des `instanceof` pour choisir un comportement que chaque type pourrait porter lui-même. Si chaque type sait faire l'opération, une méthode de l'interface et le polymorphisme (§5.3) sont préférables. `instanceof` sert quand l'opération est **extérieure** aux types. Et quand la liste des types est **fermée**, `sealed` + `switch` (§5.7) fait vérifier par le compilateur qu'aucun cas n'est oublié.

5.7 🔵 `sealed interface` et `switch` avec motifs

Avec une interface ordinaire, n'importe qui peut ajouter une implémentation. Parfois, on veut l'inverse : une liste **fermée** et connue, comme un `enum`, mais où chaque cas porte ses propres données. C'est le rôle d'une interface `sealed` (Java 17), combinée au **pattern matching dans `switch`** (Java 21) et aux **motifs de record** (Java 21).

```
// Fichier : SealedDiscount.java
sealed interface Discount permits PercentageDiscount, FixedDiscount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

/** Libellé affiché sur le ticket : le switch déstructure chaque record. */
static String label(Discount d) {
    return switch (d) {
        case PercentageDiscount(int p) when p >= 50 -> "Super promo -" + p + " %";
        case PercentageDiscount(int p)              -> "-" + p + " %";
        case FixedDiscount(BigDecimal amount)       -> "-" + amount + " €";
    };   // pas de default : le compilateur sait que la liste est complète
}

void main() {
    IO.println(label(new PercentageDiscount(10)));                // -10 %
    IO.println(label(new PercentageDiscount(50)));                // Super promo -50 %
    IO.println(label(new FixedDiscount(new BigDecimal("5.00")))); // -5.00 €
}
```

Ce qu'il faut retenir :

- `sealed … permits A, B` : seules `A` et `B` peuvent implémenter l'interface. Les records sont implicitement `final`, ce qui satisfait la règle.
- `case PercentageDiscount(int p)` : **motif de record**. Il teste le type **et** extrait le composant dans `p`.
- `when` : une **garde**, une condition supplémentaire. L'ordre des `case` compte : le plus spécifique d'abord.
- Comme pour un `enum`, le `switch` est **exhaustif** sans `default` : ajouter un troisième record à `permits` fait échouer la compilation de `label` tant que le nouveau cas n'est pas traité.

Quand choisir quoi ? Polymorphisme classique (`d.apply(total)`) quand **chaque type sait faire** l'opération. `sealed` + `switch` quand l'opération est **extérieure** aux types (affichage, export, statistiques) et qu'on veut que le compilateur vérifie qu'aucun cas n'est oublié.

---

6. Collections

6.1 🟢 `List`, `Set`, `Map` sont des interfaces

Le problème. Un tableau (S1) a une taille fixe : impossible d'ajouter un 11e plat à un `Dish[10]`. Il n'offre ni recherche, ni suppression, ni unicité.

La solution. Le *Collections Framework* (`java.util`) fournit des structures qui grandissent seules. Et maintenant que vous connaissez les interfaces, sa structure s'éclaire : **`List`, `Set` et `Map` sont des interfaces**, chacune avec plusieurs implémentations.

```
Collection<E>                       Map<K, V>  (à part : des paires clé → valeur)
 ├── List<E>  → ArrayList            ├── HashMap
 │             (LinkedList)          ├── LinkedHashMap
 ├── Set<E>   → HashSet              └── TreeMap
 │             (LinkedHashSet, TreeSet)
 └── Queue<E> → ArrayDeque
```

Interface	Garantit	Implémentation par défaut	FoodFast
`List<E>`	ordre d'insertion, accès par indice, doublons permis	`ArrayList`	le menu d'un restaurant
`Set<E>`	**pas de doublon**	`HashSet`	les types de cuisine proposés
`Map<K, V>`	une valeur par **clé** unique	`HashMap`	commandes par id, quantité par plat

La bonne pratique : déclarer l'interface, instancier l'implémentation.

```
List<Dish> menu = new ArrayList<>();          // ✅ comme Discount d = new FixedDiscount(...)
ArrayList<Dish> menu2 = new ArrayList<>();    // ❌ (déconseillé) lie le code à une implémentation
```

Avec la première forme, passer à une autre implémentation ne change qu'une ligne : le reste du code ne connaît que le contrat `List`. Les paramètres et les types de retour de vos méthodes doivent, eux aussi, être des interfaces : `void printMenu(List<Dish> menu)`.

6.2 🟢 Utiliser les génériques

`List<Dish>` se lit « liste de `Dish` ». Le type entre chevrons est un **paramètre de type** : il indique au compilateur ce que contient la collection.

```
// Fichier : GenericsUsage.java
record Dish(String name) {}

void main() {
    List<Dish> menu = new ArrayList<>();      // <> : Java déduit <Dish> (opérateur « diamant »)
    menu.add(new Dish("Margherita"));
    // menu.add("Burger");                    // ❌ erreur de compilation : String n'est pas un Dish

    Dish first = menu.get(0);                 // pas de conversion : le type est connu
    IO.println(first.name());

    List<Integer> quantities = new ArrayList<>();   // pas de List<int> : un objet est requis
    quantities.add(3);                        // autoboxing : int 3 → Integer 3
    int q = quantities.get(0);                // unboxing : Integer → int
    IO.println(q);

    Map<String, List<Dish>> menusByRestaurant = new HashMap<>();   // les génériques s'imbriquent
    menusByRestaurant.put("Pizzeria Roma", menu);
    IO.println(menusByRestaurant.get("Pizzeria Roma").size());     // 1
}
```

Ce qu'il faut retenir :

- Le compilateur **vérifie** le type des éléments : une erreur de type est détectée à la compilation, pas en production.
- Les génériques n'acceptent que des **objets** : `List<Integer>`, `Map<Dish, Integer>`, jamais `List<int>`. Java convertit automatiquement `int` ↔ `Integer` (*autoboxing*).

> **Piège** — Le type brut `List menu = new ArrayList();` (sans chevrons) compile encore, pour compatibilité avec Java 1.4, mais désactive toute vérification. ✅ Toujours préciser le type.

> **Piège** — L'unboxing d'un `null` : `int q = map.get(cle);` lève une `NullPointerException` si la clé est absente. Voir `getOrDefault` en §6.5.

6.3 🟢 `List`

Documentation : `java.util.List`

```java
// Fichier : MenuList.java
enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
    Dish burger = new Dish("Burger", new BigDecimal("9.90"), DishSize.LARGE);
    Dish salad = new Dish("César", new BigDecimal("8.00"), DishSize.SMALL);

    List<Dish> menu = new ArrayList<>();
    menu.add(pizza);
    menu.add(burger);
    menu.add(salad);

    IO.println(menu.size());              // 3
    IO.println(menu.get(1).name());       // Burger : indices à partir de 0
    IO.println(menu.getFirst().name());   // Margherita (Java 21)
    IO.println(menu.getLast().name());    // César (Java 21)
    IO.println(menu.contains(burger));    // true (utilise equals)

    menu.remove(burger);
    for (Dish d : menu) {                 // for-each, comme sur un tableau
        IO.println(d.name() + " : " + d.price() + " €");
    }

    List<DishSize> sizes = List.of(DishSize.SMALL, DishSize.LARGE);   // non modifiable
    IO.println(sizes);                    // [SMALL, LARGE]
}
```

Méthode	Effet
`add(e)` / `add(i, e)`	ajoute à la fin / à l'indice `i`
`get(i)`	élément d'indice `i`
`getFirst()`, `getLast()`	premier, dernier (Java 21, *sequenced collections*)
`remove(e)` / `remove(i)`	retire un élément / l'élément d'indice `i`
`contains(e)`, `indexOf(e)`	recherche (avec `equals`)
`size()`, `isEmpty()`	taille
`List.of(…)`	liste **non modifiable**
`List.copyOf(c)`	copie **non modifiable** d'une collection

Piège — `List.of` crée une liste **non modifiable** : `add`, `remove`, `set` lèvent une `UnsupportedOperationException`.

```java
List<DishSize> sizes = List.of(DishSize.SMALL, DishSize.LARGE);
sizes.add(DishSize.MEDIUM);   // ❌ UnsupportedOperationException
```

✅ Pour une liste modifiable : `new ArrayList<>(List.of(...))`.

Piège — `menu.get(menu.size())` : indice hors bornes (`IndexOutOfBoundsException`). Le dernier indice est `size() - 1`, ou mieux : `getLast()`.

6.4 🟢 `Set`

Un `Set` garantit l'**unicité** : ajouter un élément déjà présent (au sens d'`equals`) ne fait rien.

```
// Fichier : CuisineSet.java
void main() {
    Set<String> cuisines = new HashSet<>();
    IO.println(cuisines.add("Italienne"));   // true : ajouté
    IO.println(cuisines.add("Japonaise"));   // true
    IO.println(cuisines.add("Italienne"));   // false : déjà présent
    IO.println(cuisines.size());             // 2
    IO.println(cuisines.contains("Japonaise"));   // true, en temps constant

    // Usage typique : dédoublonner une liste
    List<String> fromOrders = List.of("Italienne", "Libanaise", "Italienne");
    Set<String> distinct = new HashSet<>(fromOrders);
    IO.println(distinct.size());             // 2
}
```

Piège — Un `HashSet` **ne garantit aucun ordre** : l'ordre d'affichage peut changer d'une version de Java à l'autre. Si l'ordre compte, utilisez `LinkedHashSet` (ordre d'insertion) ou `TreeSet` (ordre trié) — voir §6.10.

6.5 🟢 `Map` : `merge`, `getOrDefault`

Une `Map<K, V>` associe à chaque **clé** unique une **valeur**. Dans FoodFast, on en utilise deux :

- `Map<String, Order>` dans `DeliveryPlatform` : retrouver une commande par son id ;
- `Map<Dish, Integer>` dans `Order` : la quantité de chaque plat. Ici, `Dish` **est une clé** : il doit avoir un `equals` / `hashCode` correct (§6.6).

```java
// Fichier : OrderItems.java
enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
    Dish burger = new Dish("Burger", new BigDecimal("9.90"), DishSize.LARGE);

    Map<Dish, Integer> items = new LinkedHashMap<>();   // comme dans Order (ordre
d'ajout)

    items.merge(pizza, 2, Integer::sum);    // absente : pizza → 2
    items.merge(pizza, 1, Integer::sum);    // présente : 2 + 1 → 3
    items.merge(burger, 1, Integer::sum);   // burger → 1

    IO.println(items.get(pizza));                       // 3
    IO.println(items.getOrDefault(new Dish("César",
            new BigDecimal("8.00"), DishSize.SMALL), 0));   // 0, et non null
    IO.println(items.containsKey(burger));              // true

    BigDecimal total = BigDecimal.ZERO;
    for (Map.Entry<Dish, Integer> e : items.entrySet()) {
        BigDecimal qty = BigDecimal.valueOf(e.getValue());
        total = total.add(e.getKey().price().multiply(qty));
        IO.println(e.getKey().name() + " x" + e.getValue());
    }
    IO.println("Total : " + total);                     // 47.40
}
```

Méthode	Effet
`put(k, v)`	associe `v` à `k` (remplace l'ancienne valeur)
`get(k)`	valeur associée, ou **`null`** si absente
`getOrDefault(k, d)`	valeur associée, ou `d` si absente
`merge(k, v, f)`	absente : `put(k, v)` ; présente : combine l'ancienne et `v` avec `f`
`containsKey(k)`, `remove(k)`	test, suppression
`keySet()`, `values()`, `entrySet()`	parcours des clés, des valeurs, des paires

`Integer::sum`. Le troisième argument de `merge` est « la façon de combiner l'ancienne et la nouvelle valeur ». `Integer::sum` désigne la méthode `Integer.sum(a, b)`, qui renvoie `a + b`. Cette notation (une *référence de méthode*) sera expliquée en S3 ; pour l'instant, lisez-la comme « additionner ».

> **Piège** — `get` puis test de `null` puis `put` : trois lignes, et une `NullPointerException` si l'on oublie le test.

```
int current = items.get(pizza);   // ❌ NullPointerException si pizza est absente (unboxing de null)
items.put(pizza, current + 1);
```

✅ `items.merge(pizza, 1, Integer::sum);` ou `items.getOrDefault(pizza, 0)`.

```
// Fichier : UnboxingNull.java (exception attendue)
void main() {
    Map<String, Integer> stock = new HashMap<>();
    stock.put("Margherita", 5);
    IO.println(stock.getOrDefault("Burger", 0));   // ✅ 0
    int burgers = stock.get("Burger");             // ❌ NullPointerException
    IO.println(burgers);
}
```

6.6 🟢 Le contrat `equals` / `hashCode` / `toString`

Le problème. Revenons à la classe `Dish` de la partie 1, écrite comme une classe ordinaire (nous l'appelons ici `PlainDish` pour la distinguer du record).

```
// Fichier : LostDish.java
class PlainDish {               // une classe, PAS un record
    private final String name;
    PlainDish(String name) { this.name = name; }
}

void main() {
    Set<PlainDish> favorites = new HashSet<>();
    favorites.add(new PlainDish("Margherita"));

    IO.println(favorites.contains(new PlainDish("Margherita")));   // false !
    favorites.add(new PlainDish("Margherita"));
    IO.println(favorites.size());                                   // 2 : un doublon
}
```

Pourquoi. `PlainDish` hérite d'`Object` les versions par défaut d'`equals` et `hashCode`, fondées sur l'**identité** : deux `new` = deux objets différents, même avec le même nom.

Comment fonctionne un `HashSet` (ou une `HashMap`). Il range les éléments dans des « tiroirs » (*buckets*) :

1. il calcule `hashCode()` de l'élément pour choisir le tiroir ;
2. dans ce tiroir, il compare avec `equals()` pour trouver l'élément exact.

```
add(margherita1)      hashCode = 4521  → tiroir 3 : [margherita1]
contains(margherita2) hashCode = 9876  → tiroir 7 : []  → false
                      (identité différente, donc hashCode différent : mauvais tiroir)
```

La solution : redéfinir `equals` ET `hashCode`.

```
// Fichier : FixedPlainDish.java
class PlainDish {
    private final String name;
    private final BigDecimal price;

    PlainDish(String name, BigDecimal price) {
        this.name = Objects.requireNonNull(name);
        this.price = price.setScale(2, RoundingMode.HALF_UP);   // voir §3.3
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // même objet
        if (!(o instanceof PlainDish other)) return false;   // null ou autre type
        return name.equals(other.name) && price.equals(other.price);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, price);                    // MÊMES champs qu'equals
    }

    @Override
    public String toString() {
        return "PlainDish[name=" + name + ", price=" + price + "]";
    }
}

void main() {
    Set<PlainDish> favorites = new HashSet<>();
    favorites.add(new PlainDish("Margherita", new BigDecimal("12.5")));
    IO.println(favorites.contains(new PlainDish("Margherita", new BigDecimal("12.50"))));
// true
    IO.println(favorites);   // [PlainDish[name=Margherita, price=12.50]]
}
```

Lecture de `equals` :

- `this == o` : raccourci si c'est le même objet.
- `o instanceof PlainDish other` : vérifie que `o` est un `PlainDish` (et pas `null`), et le nomme `other` (*pattern matching* d'`instanceof`, Java 16).
- puis on compare les champs significatifs.

Le contrat (Javadoc d'`Object.equals`) :

1. **Réflexif** : `a.equals(a)`.
2. **Symétrique** : `a.equals(b)` ⇔ `b.equals(a)`.
3. **Transitif** : `a.equals(b)` et `b.equals(c)` ⇒ `a.equals(c)`.
4. **Cohérent** : le résultat ne change pas tant que les objets ne changent pas.
5. `a.equals(null)` vaut `false`.

6. Et surtout : `a.equals(b)` ⇒ `a.hashCode() == b.hashCode()`. L'inverse n'est pas exigé (deux objets différents peuvent avoir le même `hashCode`).

> **Piège** — Redéfinir `equals` sans `hashCode` : deux objets « égaux » tombent dans des tiroirs différents, et le `HashSet` contient des doublons. Toujours **les deux ensemble**, sur **les mêmes champs**.

> **Piège** — `public boolean equals(PlainDish other)` (paramètre `PlainDish` au lieu d'`Object`) : c'est une **surcharge**, pas une redéfinition. Les collections appellent `equals(Object)` et ne la voient jamais. `@Override` détecte l'erreur à la compilation.

Et le `record` ? Il génère `equals`, `hashCode` et `toString` sur **tous** ses composants, en respectant le contrat. C'est la raison principale de préférer un record pour une valeur : vous ne pouvez plus oublier `hashCode`.

Et une entité comme `Order` ? Une commande est définie par son **identité**, pas par son contenu : deux commandes avec les mêmes plats sont deux commandes différentes. Deux options correctes :

- ne pas redéfinir `equals` / `hashCode` (égalité d'identité, celle d'`Object`) : c'est le choix du modèle de référence ;
- les redéfinir sur le seul champ `id`, qui est `final`.

Ne les fondez **jamais** sur un champ qui change (`status`, `items`) : voir §6.8.

6.7 🟢 `record` et `BigDecimal` : le piège de l'échelle

Le record génère `equals` avec `BigDecimal.equals`, qui compare l'échelle (§3.3). Sans normalisation, la même pizza saisie avec deux échelles donne deux clés :

```java
// Fichier : ScaleTrap.java
enum DishSize { SMALL, MEDIUM, LARGE }

record RawDish(String name, BigDecimal price, DishSize size) {}   // ❌ sans setScale

record Dish(String name, BigDecimal price, DishSize size) {       // ✅ version du modèle
    Dish {
        Objects.requireNonNull(price, "price");
        price = price.setScale(2, RoundingMode.HALF_UP);
    }
}

void main() {
    Map<RawDish, Integer> raw = new HashMap<>();
    raw.merge(new RawDish("Pizza", new BigDecimal("12.5"), DishSize.MEDIUM), 1,
Integer::sum);
    raw.merge(new RawDish("Pizza", new BigDecimal("12.50"), DishSize.MEDIUM), 1,
Integer::sum);
    IO.println("Sans setScale : " + raw.size() + " lignes");   // 2 lignes « Pizza x1 »

    Map<Dish, Integer> fixed = new HashMap<>();
    fixed.merge(new Dish("Pizza", new BigDecimal("12.5"), DishSize.MEDIUM), 1,
Integer::sum);
    fixed.merge(new Dish("Pizza", new BigDecimal("12.50"), DishSize.MEDIUM), 1,
Integer::sum);
    Dish pizza = new Dish("Pizza", new BigDecimal("12.500"), DishSize.MEDIUM);
    IO.println("Avec setScale : " + fixed.size() + " ligne, quantité "
            + fixed.get(pizza));                                  // 1 ligne, quantité 2
}
```

Le prix d'un plat peut venir d'un formulaire (`"12.5"`), d'une base de données (`12.50`), d'un calcul (`12.500`) : la normalisation dans le constructeur compact rend l'égalité fiable quelle que soit la source.

6.8 🔒 Clés mutables : l'objet perdu

> 🔒 **Sécurité** — Un `equals` / `hashCode` incorrect ne fait **pas planter** le programme : il produit des doublons silencieux, des recherches qui échouent, des quantités fausses. C'est une perte d'intégrité des données, d'autant plus dangereuse qu'elle est invisible. Cas le plus vicieux : **modifier un champ utilisé par `hashCode` d'un objet déjà rangé dans un `HashSet` ou une `HashMap`**.

```
// Fichier : MutableKey.java
class Cart {                                  // ❌ equals/hashCode sur un champ modifiable
    private String promoCode;
    Cart(String promoCode) { this.promoCode = promoCode; }
    void changePromo(String code) { this.promoCode = code; }

    @Override public boolean equals(Object o) {
        return o instanceof Cart other && Objects.equals(promoCode, other.promoCode);
    }
    @Override public int hashCode() { return Objects.hashCode(promoCode); }
}

void main() {
    Set<Cart> activeCarts = new HashSet<>();
    Cart cart = new Cart("WELCOME");
    activeCarts.add(cart);

    cart.changePromo("SUMMER");               // le hashCode change : mauvais tiroir

    IO.println(activeCarts.contains(cart));  // false : on ne retrouve plus l'objet
    IO.println(activeCarts.remove(cart));    // false : impossible de le retirer
    IO.println(activeCarts.size());          // 1 : il est toujours là, inaccessible
}
```

✅ Les clés de `Map` et les éléments de `Set` doivent être **immuables** (au moins sur les champs utilisés par `equals` / `hashCode`). C'est le cas d'un record dont les composants sont immuables (`String`, `BigDecimal`, `enum`) : c'est pourquoi `Dish` peut servir de clé dans `Order`.

6.9 🔒 Copie défensive : `getItems()`

`private` protège le **champ**, mais pas l'objet qu'il référence. Si une méthode **rend** la `Map` interne, l'appelant peut la modifier directement et contourner toutes les vérifications de `addDish`.

```java
// Fichier : LeakyOrder.java
enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

class LeakyOrder {
    private final Map<Dish, Integer> items = new LinkedHashMap<>();
    private boolean confirmed;

    public void addDish(Dish dish, int quantity) {
        if (confirmed) throw new IllegalStateException("Commande déjà validée");
        if (quantity <= 0) throw new IllegalArgumentException("Quantité invalide : " +
quantity);
        items.merge(dish, quantity, Integer::sum);
    }
    public void confirm() { confirmed = true; }
    public Map<Dish, Integer> getItems() { return items; }   // ❌ rend la Map interne
}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
    LeakyOrder order = new LeakyOrder();
    order.addDish(pizza, 2);
    order.confirm();

    // Attaque : la commande est confirmée, addDish refuserait toute modification...
    order.getItems().put(pizza, -5);         // ...mais on passe à côté
    IO.println(order.getItems());            // {Dish[name=Margherita, ...]=-5}
}
```

✅ La correction du modèle : renvoyer une **vue non modifiable**.

```java
// Fichier : SafeOrder.java (exception attendue)
enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

class SafeOrder {
    private final Map<Dish, Integer> items = new LinkedHashMap<>();

    public void addDish(Dish dish, int quantity) {
        if (quantity <= 0) throw new IllegalArgumentException("Quantité invalide : " +
quantity);
        items.merge(dish, quantity, Integer::sum);
    }
    public Map<Dish, Integer> getItems() { return Collections.unmodifiableMap(items); }
// ✅
}

void main() {
    Dish pizza = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
    SafeOrder order = new SafeOrder();
    order.addDish(pizza, 2);
    IO.println(order.getItems());            // lecture : OK
    order.getItems().put(pizza, -5);         // UnsupportedOperationException
}
```

Deux outils, deux usages :

	`Collections.unmodifiableMap(items)`	`Map.copyOf(items)`
Nature	une **vue** sur la Map interne	une **copie** figée
Coût	aucun (pas de copie)	proportionnel à la taille
Si la commande change ensuite	la vue montre les changements	la copie reste à l'ancien état
Usage	getter d'une entité (`Order.getItems()`)	photo à un instant donné ; copie à l'**entrée** d'un constructeur

La copie défensive vaut aussi **à l'entrée** : si un constructeur reçoit une liste, il doit la copier (`this.dishes = List.copyOf(dishes);`), sinon l'appelant garde un accès à la liste interne.

> 🔒 **Sécurité** — Exposer un état interne modifiable est une faille d'encapsulation classique : SEI CERT *OBJ05-J. Do not return references to private mutable class members* et *OBJ06-J. Defensively copy mutable inputs and mutable internal components*. Pour un profil cybersécurité : c'est le même raisonnement qu'un contrôle fait côté client et contourné en appelant l'API directement.

6.10 🔵 Choisir son implémentation : `LinkedHashMap`, `TreeMap`, `ArrayDeque`…

La notation O(…) indique comment le temps d'une opération évolue avec le nombre *n* d'éléments : O(1) = temps constant, O(log *n*) = augmente très lentement (≈ 20 étapes pour un million d'éléments), O(*n*) = proportionnel à la taille.

Implémentation	Ordre de parcours	`get` / `contains` / `add`	Quand l'utiliser
`ArrayList`	insertion	`get(i)` O(1), `contains` O(*n*), `add` en fin O(1) amorti	**par défaut** pour une liste
`LinkedList`	insertion	`get(i)` O(*n*)	presque jamais (voir `ArrayDeque`)
`ArrayDeque`	insertion	ajout/retrait aux deux bouts O(1)	file d'attente, pile
`HashSet` / `HashMap`	**aucun** garanti	O(1) en moyenne	**par défaut** pour un ensemble, une table
`LinkedHashSet` / `LinkedHashMap`	insertion	O(1) en moyenne	ordre d'ajout voulu (ex. `Order.items`)
`TreeSet` / `TreeMap`	**trié** par clé	O(log *n*)	parcours trié, plus petite/plus grande clé

```java
// Fichier : MapOrders.java
void main() {
    Map<String, Integer> linked = new LinkedHashMap<>();
    Map<String, Integer> tree = new TreeMap<>();
    String[] dishes = { "Tiramisu", "Burger", "Margherita" };
    for (String d : dishes) {
        linked.put(d, d.length());
        tree.put(d, d.length());
    }
    IO.println(linked);   // {Tiramisu=8, Burger=6, Margherita=10} : ordre d'ajout
    IO.println(tree);     // {Burger=6, Margherita=10, Tiramisu=8} : ordre alphabétique

    TreeMap<Integer, String> byMinutes = new TreeMap<>();
    byMinutes.put(30, "order-a");
    byMinutes.put(12, "order-b");
    byMinutes.put(45, "order-c");
    IO.println(byMinutes.firstKey());        // 12 : la livraison la plus proche
    IO.println(byMinutes.headMap(31));       // {12=order-b, 30=order-a} : moins de 31 min

    Deque<String> kitchenQueue = new ArrayDeque<>();   // file d'attente de la cuisine
    kitchenQueue.addLast("order-a");
    kitchenQueue.addLast("order-b");
    IO.println(kitchenQueue.pollFirst());    // order-a : premier arrivé, premier servi
}
```

Pourquoi `Order` utilise une `LinkedHashMap` : le ticket affiche les plats **dans l'ordre où le client les a ajoutés**. Avec une `HashMap`, l'ordre serait imprévisible.

> **Piège** — Un `TreeMap` / `TreeSet` utilise `compareTo` (et non `equals`) pour décider si deux clés sont identiques. Pour un `TreeSet<BigDecimal>`, `12.5` et `12.50` sont donc **le même** élément, alors qu'ils sont deux éléments distincts dans un `HashSet<BigDecimal>`.

6.11 🟣 Écrire ses propres génériques

Vous avez utilisé `List<Dish>`. Vous pouvez aussi écrire vos propres classes et méthodes paramétrées par un type. Exemple : un dépôt en mémoire réutilisable pour les commandes, les clients, les restaurants.

```java
// Fichier : InMemoryRepository.java
/** K : type de la clé, V : type de la valeur. Le même code sert pour tous les types. */
class InMemoryRepository<K, V> {
    private final Map<K, V> storage = new HashMap<>();

    void save(K id, V value) {
        storage.put(Objects.requireNonNull(id), Objects.requireNonNull(value));
    }

    Optional<V> findById(K id) {
        return Optional.ofNullable(storage.get(id));
    }

    int count() { return storage.size(); }
}

/** Méthode générique : <T> se déclare avant le type de retour. */
static <T> T lastOr(List<T> list, T fallback) {
    return list.isEmpty() ? fallback : list.getLast();
}

/** Borne : T doit être comparable à lui-même (String, BigDecimal, Integer...). */
static <T extends Comparable<T>> T max(List<T> values) {
    T best = values.getFirst();
    for (T v : values) {
        if (v.compareTo(best) > 0) best = v;
    }
    return best;
}

record Customer(String name) {}

void main() {
    InMemoryRepository<String, Customer> customers = new InMemoryRepository<>();
    customers.save("c-1", new Customer("Alice"));
    IO.println(customers.findById("c-1").orElseThrow().name());   // Alice
    IO.println(customers.findById("c-2").isPresent());            // false

    IO.println(lastOr(List.of("a", "b"), "vide"));                // b
    IO.println(lastOr(List.<String>of(), "vide"));                // vide

    List<BigDecimal> prices = List.of(new BigDecimal("9.90"), new BigDecimal("12.50"));
    IO.println(max(prices));                                      // 12.50
}
```

Le compilateur vérifie les types à chaque usage : `customers.save("c-1", "Alice")` ne compile pas (une `String` n'est pas un `Customer`). Pour aller plus loin (jokers `? extends`, `? super`) : tutoriel dev.java — Generics.

6.12 🔵 `Deque` : une file à deux bouts

Pourquoi ? Une file d'attente (premier arrivé, premier servi), une pile (dernier arrivé, premier servi), ou une liste dans laquelle on ajoute et retire **aux deux extrémités** : avec une `ArrayList`, retirer le premier

élément décale tous les autres (O(*n*)). L'interface `Deque` (*double-ended queue*, « file à deux bouts ») est faite pour cela, et son implémentation `ArrayDeque` fait chaque ajout ou retrait aux extrémités en O(1).

Opération	Au début	À la fin
Ajouter	`addFirst(e)`	`addLast(e)`
Retirer et renvoyer (ou `null` si vide)	`pollFirst()`	`pollLast()`
Regarder sans retirer (ou `null` si vide)	`peekFirst()`	`peekLast()`

S'y ajoutent `size()`, `isEmpty()`, le parcours for-each (du premier au dernier) et `stream()`.

Exemple concret : la file des commandes en cuisine. Les commandes normales arrivent à la fin ; une commande prioritaire (client premium) passe devant.

```
// Fichier : KitchenDeque.java
void main() {
    Deque<String> kitchen = new ArrayDeque<>();
    kitchen.addLast("order-1");
    kitchen.addLast("order-2");
    kitchen.addFirst("order-vip");                // passe devant

    IO.println(kitchen);                          // [order-vip, order-1, order-2]
    IO.println(kitchen.size());                   // 3
    IO.println(kitchen.pollFirst());              // order-vip : préparée en premier
    IO.println(kitchen.pollLast());               // order-2   : la dernière arrivée,
annulée
    IO.println(kitchen.peekFirst());              // order-1   : regardée, pas retirée
    IO.println(kitchen.size());                   // 1

    kitchen.pollFirst();
    IO.println(kitchen.pollFirst());              // null : file vide, pas d'exception

    // Lire une « tranche » : les 2 premières commandes, sans modifier la file
    kitchen.addLast("order-3"); kitchen.addLast("order-4"); kitchen.addLast("order-5");
    List<String> firstTwo = kitchen.stream().limit(2).toList();   // streams : séance 3
    IO.println(firstTwo);                         // [order-3, order-4]
}
```

Piège.

❌ `ArrayDeque` n'a pas d'accès par indice : pas de `get(i)`. Pour lire l'élément d'indice `i`, il faut parcourir.

❌ `ArrayDeque` refuse `null` (`addLast(null)` lève une `NullPointerException`) : `null` sert justement à signaler « file vide » dans `pollFirst`.

❌ `removeFirst()` et `getFirst()` lèvent une `NoSuchElementException` sur une file vide, alors que `pollFirst()` et `peekFirst()` renvoient `null`. Choisissez en connaissance de cause.

> 🔒 **Sécurité** — Une classe qui encapsule une `Deque` ne doit pas la renvoyer telle quelle : l'appelant pourrait la vider ou la remplir sans passer par vos règles. Renvoyez une copie (`List.copyOf(deque)`) ou une vue non modifiable, comme pour `getItems()` (§6.9).

---

7. `Optional`

7.1 🟢 Pourquoi `Optional`

Le problème. `Map.get` renvoie `null` quand la clé est absente. Si une méthode de recherche fait de même, **rien dans sa signature ne le dit** : l'appelant oublie de tester, et la `NullPointerException` survient plus loin, loin de la cause.

```
// ❌ la signature promet une Order... qui peut être null
public Order findOrderById(String id) {
    return orders.get(id);
}

Order o = platform.findOrderById("order-inconnu");
IO.println(o.getStatus());   // NullPointerException, ici ou trois méthodes plus loin
```

La solution. `Optional<Order>` est une « boîte » qui contient une `Order` ou rien. Le type de retour **dit** que l'absence est possible, et l'appelant doit la traiter explicitement pour obtenir la valeur.

7.2 🟢 Utiliser `Optional`

Documentation : `java.util.Optional`

```java
// Fichier : FindOrder.java (exception attendue)
enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

class Order {
    private final String id = "order-" + UUID.randomUUID();
    private OrderStatus status = OrderStatus.PENDING;
    public String getId() { return id; }
    public OrderStatus getStatus() { return status; }
}

class DeliveryPlatform {
    private final Map<String, Order> orders = new HashMap<>();
    public void placeOrder(Order order) { orders.put(order.getId(), order); }
    public Optional<Order> findOrderById(String id) { return
Optional.ofNullable(orders.get(id)); }
}

void main() {
    DeliveryPlatform platform = new DeliveryPlatform();
    Order order = new Order();
    platform.placeOrder(order);

    Optional<Order> found = platform.findOrderById(order.getId());
    IO.println(found.isPresent());                              // true

    Optional<Order> missing = platform.findOrderById("order-42");
    if (missing.isEmpty()) {
        IO.println("Commande inconnue");                        // affiché
    }

    Order sure = platform.findOrderById(order.getId()).orElseThrow();   // je sais qu'elle
existe
    IO.println(sure.getStatus());                               // PENDING

    platform.findOrderById("order-42").orElseThrow();           // NoSuchElementException
}
```

Méthode	Rôle
`Optional.of(x)`	boîte pleine (`x` ne doit pas être `null`)
`Optional.ofNullable(x)`	pleine si `x != null`, vide sinon : idéal autour de `Map.get`
`Optional.empty()`	boîte vide
`isPresent()`, `isEmpty()`	test
`orElseThrow()`	la valeur, ou une `NoSuchElementException` immédiate et claire
`orElse(defaut)`	la valeur, ou une valeur par défaut

`orElseThrow()` se lit « je m'attends à ce que la valeur existe ; sinon, c'est une erreur ». L'erreur survient **à l'endroit du problème**, avec un message explicite, au lieu d'une `NullPointerException` plus loin. En

S3, vous verrez comment lever votre propre exception (`OrderNotFoundException`) et les méthodes `map`, `ifPresentOrElse`, qui prennent des lambdas.

7.3 🟢 Les pièges d'`Optional`

Piège 1 — `get()` sans vérifier. `opt.get()` sur une boîte vide lève une `NoSuchElementException` : on a remplacé une exception par une autre, sans rien gagner. Les concepteurs de l'API eux-mêmes le déconseillent.

```
Order o = platform.findOrderById(id).get();           // ❌
Order o = platform.findOrderById(id).orElseThrow();   // ✅ même effet, intention explicite
```

Piège 2 — `Optional` en champ, en paramètre ou dans une collection. `Optional` est conçu pour les **types de retour**.

```
private Optional<String> phone;              // ❌ champ : utilisez null en interne, ou une valeur par défaut
void notify(Optional<String> phone) { }      // ❌ paramètre : faites deux méthodes, ou acceptez null et vérifiez
List<Optional<Order>> orders;                // ❌ une liste contient des éléments, ou est vide
```

Piège 3 — Renvoyer `null` depuis une méthode qui renvoie `Optional`. L'appelant ne testera pas : c'est le pire des deux mondes.

```
public Optional<Order> findOrderById(String id) {
    if (id == null) return null;              // ❌
    return Optional.ofNullable(orders.get(id));
}
// ✅ return Optional.empty();
```

Piège 4 — `Optional` pour une collection. Une méthode qui cherche **plusieurs** résultats renvoie une **liste vide**, jamais `Optional<List<…>>` ni `null`.

```java
// Fichier : OptionalTraps.java
void main() {
    Map<String, String> phones = new HashMap<>();
    phones.put("alice", "06 12 34 56 78");

    Optional<String> alicePhone = Optional.ofNullable(phones.get("alice"));
    Optional<String> bobPhone = Optional.ofNullable(phones.get("bob"));

    IO.println(alicePhone.orElse("pas de numéro"));   // 06 12 34 56 78
    IO.println(bobPhone.orElse("pas de numéro"));     // pas de numéro
    IO.println(bobPhone.isEmpty());                   // true

    List<String> noResult = List.of();                // ✅ « aucun résultat » = liste
vide
    IO.println(noResult.size());                      // 0
}
```

---

8. Le modèle FoodFast complet

Voici le modèle construit pendant la séance, identique au modèle de référence du cours (`cours/modele/FoodFastModel.java`). Chaque élément renvoie à la partie où il a été expliqué.

Élément	Notion	Partie
`DishSize`, `OrderStatus`	`enum`	§2.1
`Dish`	`record`, constructeur compact, `BigDecimal`, `setScale`	§2.4, §2.5, §3.3
`User`, `Customer`, `Courier`	classe abstraite, `extends`, `super`, `@Override`	§4
`Order`	encapsulation, méthodes métier, `Map`, `merge`, copie défensive	§1.3, §1.4, §6.5, §6.9
`Discount`, `PercentageDiscount`, `FixedDiscount`	interface, polymorphisme	§5
`DeliveryPlatform`	`Map<String, Order>`, `Optional`	§6.5, §7

```java
// Fichier : FoodFastModel.java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDateTime;
import java.util.*;

enum DishSize { SMALL, MEDIUM, LARGE }

enum OrderStatus { PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED }

/** Valeur immuable : un plat du menu. */
record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        if (name.isBlank()) throw new IllegalArgumentException("Nom vide");
        if (price.signum() < 0) throw new IllegalArgumentException("Prix négatif : " +
price);
        price = price.setScale(2, RoundingMode.HALF_UP); // 12.5 et 12.50 deviennent égaux
    }
}

/** Héritage : ce que partagent tous les utilisateurs de la plateforme. */
abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;

    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public abstract String role();
}

class Customer extends User {
    private final String address;
    Customer(String name, String email, String address) {
        super(name, email);
        this.address = Objects.requireNonNull(address);
    }
    public String getAddress() { return address; }
    @Override public String role() { return "CLIENT"; }
}

class Courier extends User {
    Courier(String name, String email) { super(name, email); }
    @Override public String role() { return "LIVREUR"; }
}

/** Entité mutable, encapsulée : une commande. */
class Order {
```

```
    private final String id = "order-" + UUID.randomUUID();
    private final Customer customer;
    private final Map<Dish, Integer> items = new LinkedHashMap<>();
    private final LocalDateTime createdAt = LocalDateTime.now();
    private OrderStatus status = OrderStatus.PENDING;

    Order(Customer customer) { this.customer = Objects.requireNonNull(customer); }

    public void addDish(Dish dish, int quantity) {
        if (status != OrderStatus.PENDING) throw new IllegalStateException("Commande déjà
validée");
        if (quantity <= 0) throw new IllegalArgumentException("Quantité invalide : " +
quantity);
        items.merge(dish, quantity, Integer::sum);
    }

    public BigDecimal getTotal() {
        BigDecimal total = BigDecimal.ZERO;
        for (Map.Entry<Dish, Integer> e : items.entrySet()) {
            total =
total.add(e.getKey().price().multiply(BigDecimal.valueOf(e.getValue())));
        }
        return total;
    }

    public void confirm() {
        if (status != OrderStatus.PENDING || items.isEmpty()) throw new
IllegalStateException("Confirmation impossible");
        status = OrderStatus.CONFIRMED;
    }

    public String getId() { return id; }
    public Customer getCustomer() { return customer; }
    public Map<Dish, Integer> getItems() { return Collections.unmodifiableMap(items); }
    public OrderStatus getStatus() { return status; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}

/** Interface + polymorphisme. */
interface Discount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

record FixedDiscount(BigDecimal amount) implements Discount {
    public BigDecimal apply(BigDecimal total) { return
total.subtract(amount).max(BigDecimal.ZERO); }
}

class DeliveryPlatform {
    private final Map<String, Order> orders = new HashMap<>();
```

```
    public void placeOrder(Order order) { orders.put(order.getId(), order); }
    public Optional<Order> findOrderById(String id) { return
Optional.ofNullable(orders.get(id)); }
    public Collection<Order> getOrders() { return
Collections.unmodifiableCollection(orders.values()); }
}

public class FoodFastModel {
    public static void main(String[] args) {
        Dish pizza = new Dish("Margherita", new BigDecimal("12.5"), DishSize.MEDIUM);
        Dish same = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
        Dish burger = new Dish("Burger", new BigDecimal("9.90"), DishSize.LARGE);
        Customer alice = new Customer("Alice", "alice@example.com", "10 rue de Paris");

        Order order = new Order(alice);
        order.addDish(pizza, 2);
        order.addDish(same, 1);      // même plat : la quantité passe à 3
        order.addDish(burger, 1);
        order.confirm();

        DeliveryPlatform platform = new DeliveryPlatform();
        platform.placeOrder(order);

        Discount[] discounts = { new PercentageDiscount(10), new FixedDiscount(new
BigDecimal("5.00")) };
        IO.println("Client : " + order.getCustomer().getName() + " (" + alice.role() +
")");
        IO.println("Plats  : " + order.getItems().size() + " lignes");                 //
2 lignes
        IO.println("Total  : " + order.getTotal());                                    //
47.40
        for (Discount d : discounts) {
            IO.println("Avec " + d + " : " + d.apply(order.getTotal()));
        }
        Order found = platform.findOrderById(order.getId()).orElseThrow();
        IO.println("Statut : " + found.getStatus());                                   //
CONFIRMED
        IO.println("Inconnue ? " + platform.findOrderById("order-42").isEmpty());      //
true
    }
}
```

> 🔵 **Pour tester** — Dans un projet Maven, placez ces types dans `src/main/java` (un fichier par type public) et écrivez des tests JUnit (S1) pour chaque invariant : prix négatif refusé, `12.5` égal à `12.50`, `addDish` refusé après `confirm`, `getItems()` non modifiable.

Le record `Dish` dans son propre fichier, tel qu'il serait dans `src/main/java` :

```java
// Fichier : Dish.java (compilation seule)
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

enum DishSize { SMALL, MEDIUM, LARGE }

record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        if (name.isBlank()) throw new IllegalArgumentException("Nom vide");
        if (price.signum() < 0) throw new IllegalArgumentException("Prix négatif : " +
price);
        price = price.setScale(2, RoundingMode.HALF_UP);
    }
}
```

Et son test JUnit :

```java
// Fichier : DishTest.java (avec Dish.java)
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import static org.junit.jupiter.api.Assertions.*;

class DishTest {
    @Test
    void prixNormaliseADeuxDecimales() {
        Dish a = new Dish("Margherita", new BigDecimal("12.5"), DishSize.MEDIUM);
        Dish b = new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM);
        assertEquals(a, b);
        assertEquals(a.hashCode(), b.hashCode());
    }

    @Test
    void prixNegatifRefuse() {
        assertThrows(IllegalArgumentException.class,
                () -> new Dish("Margherita", new BigDecimal("-1"), DishSize.MEDIUM));
    }
}
```

(Le `() -> …` de `assertThrows` est une lambda : vous l'avez utilisée en S1 avec JUnit sans la détailler ; elle sera expliquée en S3.)

---

9. Ce qu'il faut retenir

Classe et objet

- Une classe regroupe des données (champs `private`) et des règles (méthodes). Le constructeur **vérifie les invariants** : un objet invalide ne doit pas pouvoir exister.
- Des **méthodes métier** (`addDish`, `confirm`) plutôt que des setters.
- `static` : appartient à la classe (constantes, utilitaires). Sans `static` : appartient à chaque objet, accès à `this`.

`enum` et `record`

- `enum` pour un ensemble fermé de valeurs ; `switch` exhaustif, sans `default`.
- `record` pour une valeur immuable : constructeur, accesseurs, `equals`, `hashCode`, `toString` générés. Le **constructeur compact** valide et normalise.

Types riches

- Montants : `BigDecimal`, construit depuis une `String`, immuable (récupérer le résultat), comparé avec `compareTo`. `equals` compare aussi l'échelle : d'où `setScale(2)` dans `Dish`.
- Dates : `java.time` (`LocalDateTime`), immuable. Identifiants : `UUID.randomUUID()` — 🔒 qui ne remplace pas un contrôle d'accès.

Héritage et interfaces

- `extends` si « est un » ; `super(...)` construit la partie mère ; `@Override` systématique ; `abstract` pour ce qui n'existe pas seul.
- Sinon, **composition** (« a un »).
- **Interface** = contrat ; **polymorphisme** = le code appelant ne connaît que le contrat. Classe abstraite seulement pour partager de l'état.

Collections

- `List` (ordre), `Set` (unicité), `Map` (clé → valeur) sont des **interfaces** : `List<Dish> menu = new ArrayList<>()`.
- `merge`, `getOrDefault`, `getFirst` / `getLast`, `List.of` (non modifiable).
- `equals` **et** `hashCode` ensemble, sur les mêmes champs, immuables. Le record le fait pour vous.

`Optional`

- En type de retour d'une recherche. `orElseThrow()`, `orElse()`, `isEmpty()`. Jamais `get()` seul, jamais en champ ou en paramètre, jamais `null`.

🔒 Sécurité

- Invariants vérifiés à la construction.
- Ne jamais exposer une collection interne modifiable : `Collections.unmodifiableMap`, `List.copyOf`.
- Un `equals` / `hashCode` incorrect perd des données en silence ; une clé de `Map` doit être immuable.
- `toString()` ne doit pas contenir de donnée sensible.

---

10. Pour aller plus loin

Documentation officielle Java 25

- `BigDecimal`, `Object.equals` et `hashCode`, `Optional`, package `java.time`
- Collections Framework — vue d'ensemble

Tutoriels dev.java

- Classes et objets, Records, Enums
- Héritage, Interfaces
- Collections, Generics

JEP (fonctionnalités vues dans cette séance)

- JEP 395 — Records, JEP 409 — Sealed Classes
- JEP 440 — Record Patterns, JEP 441 — Pattern Matching for switch
- JEP 431 — Sequenced Collections, JEP 513 — Flexible Constructor Bodies

Sécurité

- SEI CERT Oracle Coding Standard for Java : règles OBJ01-J (accessibilité des champs), OBJ05-J et OBJ06-J (copies défensives), MET09-J (`equals` et `hashCode`).
- OWASP Top 10 — A01 Broken Access Control (IDOR).

Défis 🟣

1. Rendre `Discount` `sealed` dans le modèle complet (§8), puis écrire `String receipt(Order order, Discount d)` qui affiche le ticket avec un `switch` à motifs.
2. Ajouter à `Order` une méthode `cancel()` qui n'est permise que dans les statuts `PENDING` et `CONFIRMED` (utiliser un `switch` exhaustif sur `OrderStatus`).
3. Écrire un `record Money(BigDecimal amount, Currency currency)` qui refuse d'additionner deux devises différentes, et l'utiliser à la place de `BigDecimal` dans `Dish`.
4. Écrire `Order.equals` / `hashCode` fondés sur le seul `id`, et un test JUnit qui montre qu'une commande reste retrouvable dans un `HashSet` après `addDish` et `confirm`.